

DUALPATH-CORE A MULTI-AGENT HYBRID TTRPG ENGINE

ภณลภัส สุทธิมาลา 65340500046

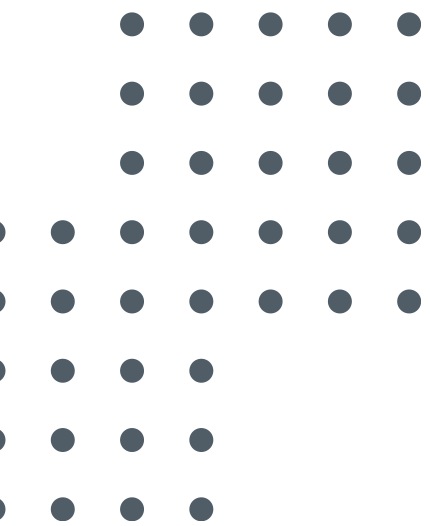
อาจารย์ที่ปรึกษา

ผศ.ดร. สุริยา นัฏสูภักพงศ์

ผศ.ดร. ไพสิฐ ขันอาสา

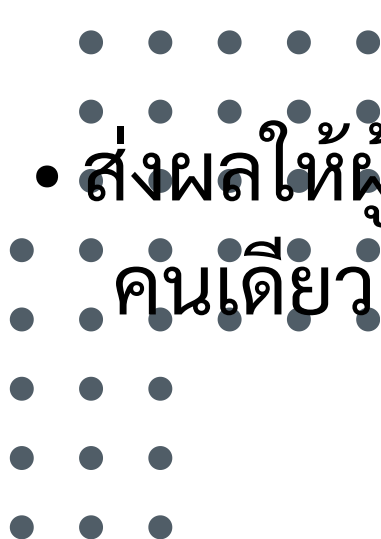


Recap



ที่มาและความสำคัญของปัญหา

High Entry Barrier

- เกม D&D มีกฎกติกาที่หนาและซับซ้อนกว่า 300+ หน้า
- จำเป็นต้องใช้ "Dungeon Master (DM)" มนุษย์ที่มีทักษะสูงในการควบคุมเกมและคำนวณสเตตัส
-  ส่งผลให้ผู้เล่นใหม่และกลุ่มเล่นคนเดียว เข้าถึงเกมนี้ได้ยาก

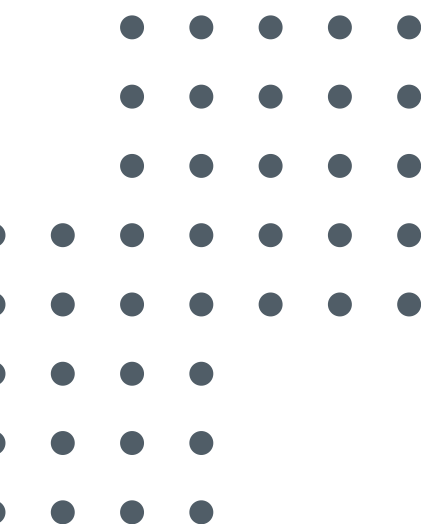
Mechanical Hallucination

- เมื่อผู้เล่นเปลี่ยนมาใช้ LLM ทั่วไปเพื่อทำหน้าที่เป็น Solo AI DM
- พื้นฐานโมเดลที่เป็น ความน่าจะเป็น ทำให้ AI เกิดอาการลืมนเล็ด ลืมเลขเต่าผิด และเสกไอเทมทะเล กฏเกณฑ์กติกา

Natural Language Variety

- [NEW PIVOT] มนุษย์ไม่ได้พิมพ์คำสั่งตายตัว แต่ป้อนอินพุตด้วยภาษามนุษย์ที่ยืดหยุ่น และมีความคลุมเครือสูง
- เอนจินโปรแกรมปกติแกะบริบทไม่ได้ ส่วนตัว LLM เดี่ยวๆ ก็จะเคลิ้มตามคำสั่งมนุษย์จนยอมให้ผู้เล่นโกงกฎแบบไร้ขอบเขต (Yes-Man Exploit)

ปัญหา + challenge ที่ อ. แนะนำเกี่ยว
กับภาษามนุษย์หลากหลาย



Progress 2 Recap & Scope Pivot

1. Old Scope (ขอบเขตเดิม)

- พัฒนา Full-stack App (มี UI/Frontend)
- มีระบบผู้เล่นหลายคน & ระบบประมวลผลเสียง (ASR/TTS)

• ปัญหา: สโคปกว้างเกินไปจนเนื้องานขาดโฟกัส



2. Committee Critique (ข้อติงจากกรรมการ)



3. Final Strategic Pivot (ขอบเขตปัจจุบัน)

- 100% Backend Focus: ตัด Frontend/UI และระบบเสียงออกทั้งหมด
- Rigorous Verification: โยกทรัพยากรมาโฟกัสที่ Architecture, Hybrid Routing และการรัน Automated Test 90 Scenarios

Exploration Scope Pivot: Open-World to Hub-Based

- ✗ Open-World Architecture (ขอบเขตเดิม)
- โลกกว้างอิสระ ไม่มีขอบเขตที่แน่นอน
 - ปัญหา: ระบบประมวลผลซับซ้อนเกิน
ความจำเป็น ทำให้ AI เกิดอาการ Context
Hallucination ได้ง่ายเมื่อเล่นเป็นเวลา
นาน

- 🎯 Hub-Based & Node Loop (ขอบเขตปัจจุบัน)
- เปลี่ยนเป็นระบบ Hub-Based (เริ่มต้นที่กิลด์
เพื่อรับเควสต์) ➡ ตัดเข้าดันเจี้ยนจำลองแบบ
Node-Based Room
 - ผลลัพธ์: ตัวเกม Loop ทำงานได้รวดเร็วขึ้น สม
เหตุสมผล และคุมรูปเกมได้นิ่งสนิท

⚙️ Engineering Justification (เหตุผลเชิงวิศวกรรม)

• • • • • โหมดการต่อสู้ (Combat Loop) คือส่วนที่มีความซับซ้อนสูงสุดในการประสานงาน
• • • • • ระหว่าง Path A (ตรรกะโค้ด) และ Path B (ความคิดสร้างสรรค์)
• • • • • และเปลี่ยนจากระบบ Open-World ให้เป็นระบบ Node ช่วยให้มีความสามารถในการพัฒนา
• • • • • และประเมินผล Core Engine นี้ ภายใต้วงเวลาที่จำกัด

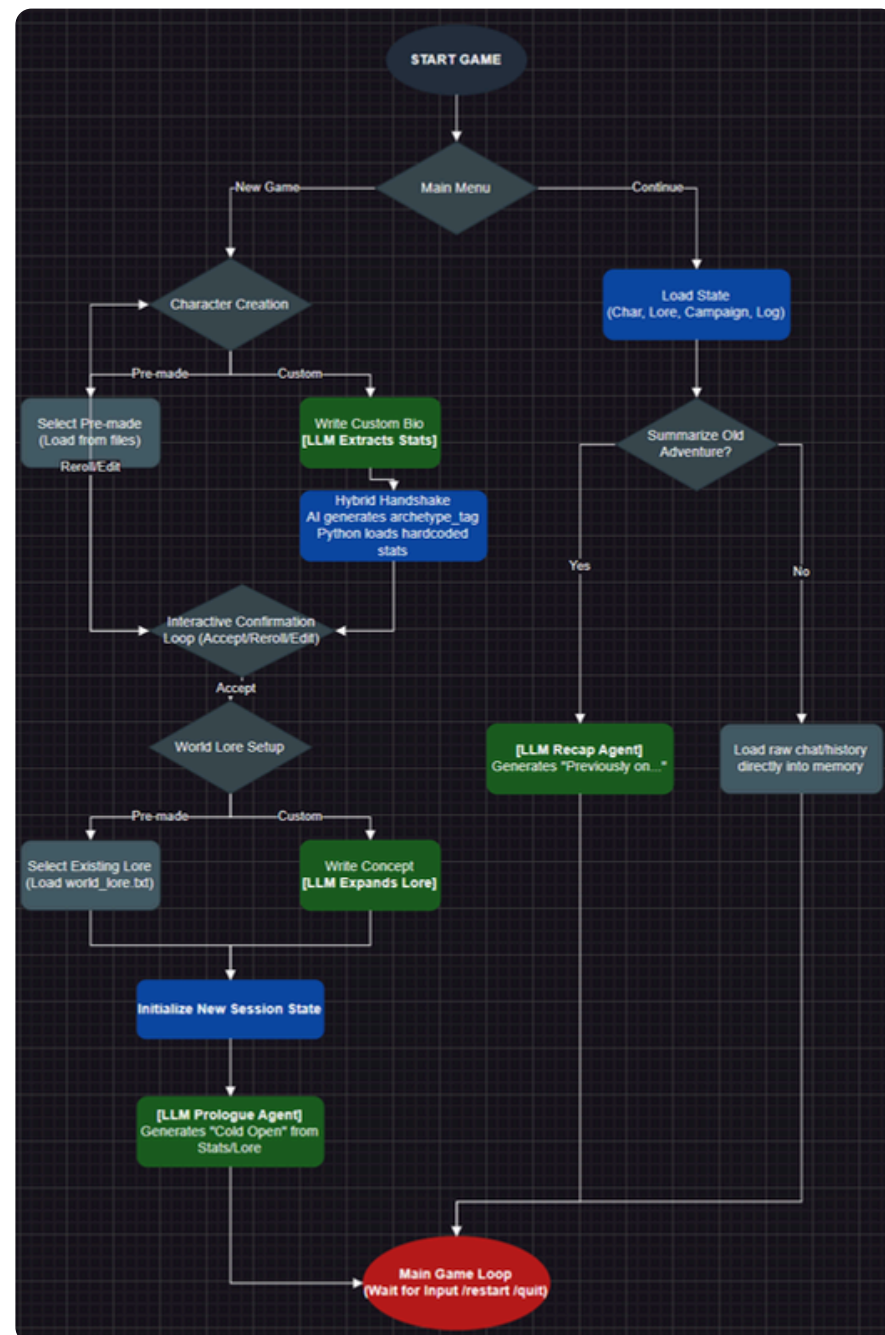
ตาราง Recap งานวิจัย

งานวิจัยที่อ้างอิง (Literature)	สิ่งที่ระบบเก่าทำได้ (Their Setup)	ข้อจำกัด / ช่องว่าง (The Gap)	Insight ที่เราหยิบมาอัปเดตและใช้เป็น Metric ในรอบนี้
Triyason (2023)	ใช้ ChatGPT ตัวเดียว คุมเกม D&D ทั้งหมด	Mechanical Hallucination: AI ลืมเลือด ลืมไอเทม	เราใช้เป็น Problem Statementหลัก: แยก Rules Engine ออกมาจาก AI วัดผลด้วยค่า S_sync
Sakellaridis (2024)	Fine-tune โมเดลด้วยบทสนทนา D&D จริง	The Fine-Tuning Paradox: AI บังคับเนื้อเรื่อง (Railroading) ผู้เล่นไม่มีอิสระ	หยิบเกณฑ์วัดผล DM Competence มาใช้: สร้างพาร์ท Path B (Arbiter) วัดผลด้วยค่า P_ground
Jørgensen et al. (2024)	ใช้ Multi-Agent (แยก Narrator และ Archivist)	High Complexity & Latency: ระบบใหญ่เกินไป หน่วงและกินพลังงานมาก	หยิบแนวคิดแยก Logic ออกจาก Narrative: ลดทอนกฎเหลือ Lite 5e วัดผลด้วยค่า C_narr
Song et al. (2024)	ใช้เทคนิค Function Calling และ RAG คุมเกม	Overusing Functions: AI เรียกใช้เครื่องมือมั่วซั่วพร่ำเพรื่อ	หยิบแนวคิด Automated Unit Tests มาใช้: สร้าง Intent Router บังคับเลน วัดผลด้วยค่า A_route

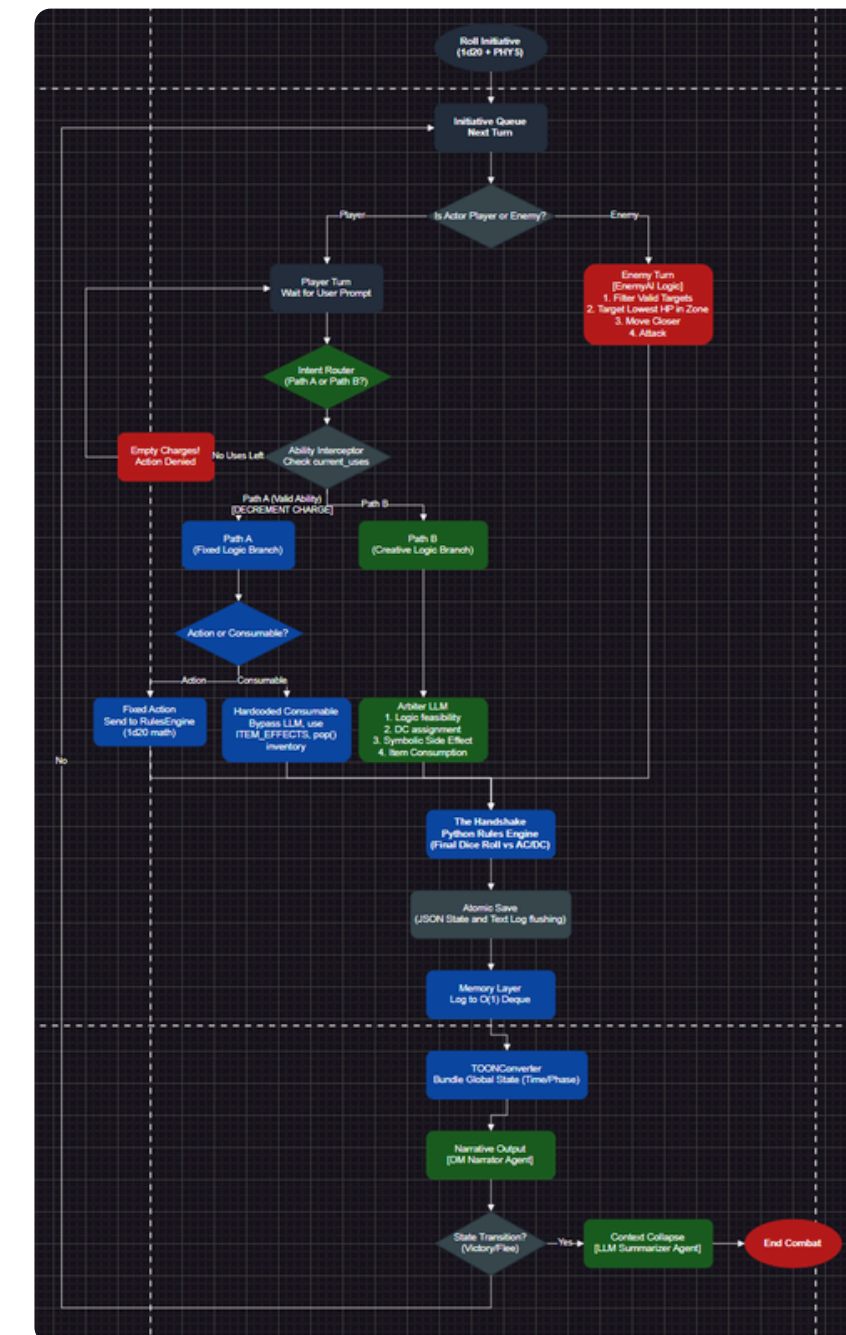
recap timeline

ลำดับ	แผนดำเนินงาน	ม.ค.	ก.พ.	มี.ค.	เม.ย	พ.ค.
3	พัฒนาเอนจินกฎเกณฑ์เชิงกำหนด (Rules Engine)					
4	พัฒนาเอเจนต์ตัดสินใจและระบบคัดกรอง (Routing)					
5	เพิ่มประสิทธิภาพและบริหารทรัพยากร					
6	ทดสอบระบบ (50-Scenario Test)					
7	ขยายระบบสู่โหมดสำรวจและเนื้อเรื่อง					
8	พัฒนา UX และสรุปผลรายงาน					
9	ทดสอบกับผู้เล่นจริงและเก็บข้อมูล					
10	วิเคราะห์ผลและจัดทำรายงาน					

สร้าง Start Menu และ Combat Menu



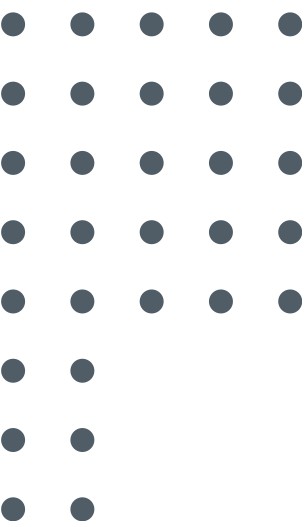
START MENU



COMBAT MENU

Core Architecture: Multi-Agent Engine Portfolio

ชื่อเอเจนต์ (Agent Name)	บทบาทหน้าที่เชิงซอฟต์แวร์ (Core Function)	รูปแบบผลลัพธ์ (TOON Syntax Output Example)
1. Intent Router	คัดแยกเจตนาผู้เล่นเป็นระบบกึ่งไม่การตัดสินใจ	type:FIXED_COMBO move_target:MID attack_target:e1
2. Action Arbiter	ตรวจสอบความสมเหตุสมผลและประเมินค่าความยาก	allowed:true check_stat:PHYS dc:12 condition:PRONE
3. Item Arbiter	อนุมานและคัดแยกประเภทไอเทมกำกวมจากชื่อ	is_consumable:true effect_type:SMALL_HEAL
4. Combat Narrator	แปลง Log ตัวเลขคณิตศาสตร์หลังบ้านเป็นบทบรรยาย	บทพากย์ภาษาธรรมชาติแบบไร้ตัวเลข (No Numbers Rule)
5. Memory Summarizer	ย่อประวัติการต่อสู้ทั้งหมดให้เหลือประโยคเดียว	"The party successfully incinerated the goblin threat."



Data Optimization & Backend Persistence

1. TOON Serialization Protocol

กลไก: บีบอัดโครงสร้าง JSON ดั้งเดิมให้เป็น TOON Syntax (hp:20|ac:16|zone:near)

ลดปริมาณ Token ลง 54% และเร่งความเร็วในการอ่านบริบทของ AI (Faster Attention)

2. Dual-Track Memory System

กลไก: แยกถังความจำระยะสั้นออกเป็น Combat Memory และ Story Memory

3. Persistence & Fault Tolerance

กลไก: ทำวงจรรบันทึกประวัติการเล่นทุกเทิร์น (Turn-based Auto-Save) ลงไฟล์ JSON ลง Local ดิสก์

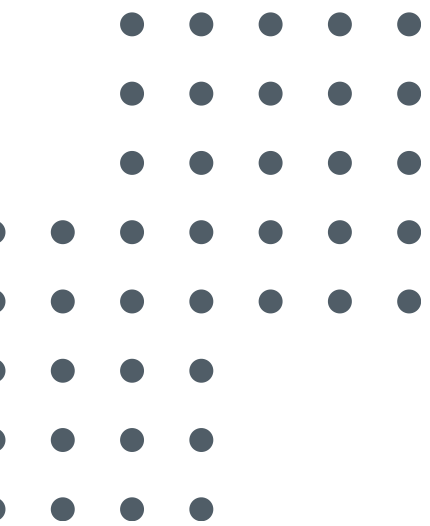
รองรับระบบ Save/Load และฟีเจอร์ Resume เล่นต่อได้แม้ระบบหลุด

4. Auto-Loot Automation

กลไก: ดึงการคำนวณไอเทมดรอปจากมอนสเตอร์กลับมาให้ Python จัดการสุ่มเมื่อตรวจเจอสถานะ DEAD

ป้องกัน AI มโนเสกไอเทมระดับเทพให้ผู้เล่น

Feedback Response



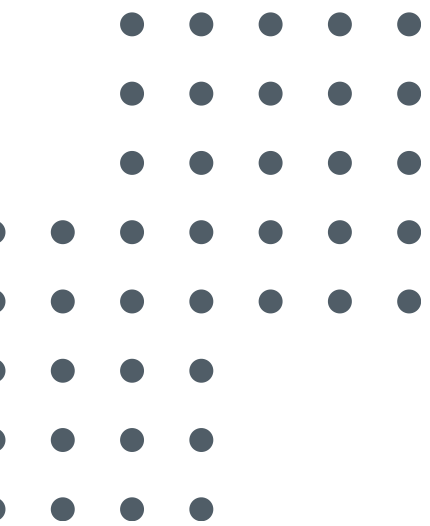
สถาปัตยกรรมและคำศัพท์เฉพาะ

ข้อเสนอแนะจากคณะกรรมการ (Feedback)	การปรับปรุงและเหตุผลเชิงวิศวกรรม (Action & Justification)
การใช้ LLM พร่ำเพรื่อ และฝืนธรรมชาติ: (ทำไมใช้ LLM ทำ Intent Router? ทำไมไม่ใช้ if-else หรือ ML ธรรมดา?)	ปรับเป็นระบบ Hybrid ชัดเจน (แบ่งหน้าที่): - LLM ทำหน้าที่แปลภาษา (Semantic Parser): เพราะคำสั่งผู้เล่นคาดเดาไม่ได้ (Infinite Input) หรือการใช้คำ synonyms การใช้ if-else จะดักคำศัพท์ได้ไม่หมด - Python ทำหน้าที่คำนวณและคุมกฎ: โค้ดรับข้อมูลจาก LLM มาคิดเลขและทยอยเต่า เพื่อความแม่นยำ 100%
ถ้าคำสั่งมีทั้ง "ตายตัว" และ "สร้างสรรค์" ผสมกัน?	เพิ่มระบบจัดการลำดับ (Action Prioritization): - Router จะแยกคำสั่ง Move และ Action ออกจากกัน - ถ้าผู้เล่นทำเกินกติกา (1 Move + 1 Action) Python Guardrail จะปฏิเสธคำสั่ง (Deny) ทันที
การใช้คำศัพท์ผิดความหมาย: (เช่น Deterministic, Single Source of Truth)	ปรับแก้คำศัพท์ใหม่ในเล่มทั้งหมด: ไม่เน้นใช้คำศัพท์ที่อลังการ แต่เน้นเข้าใจง่าย ตรงตัว

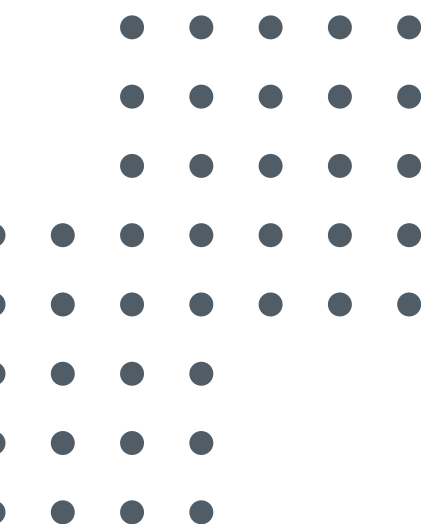
สถาปัตยกรรมและคำศัพท์เฉพาะ

ข้อเสนอแนะจากคณะกรรมการ (Feedback)	การปรับปรุงและเหตุผลเชิงวิศวกรรม (Action & Justification)
นิยามของ "Baseline" ไม่ชัดเจน และข้อมูลดูเหมือนการสุม: (Baseline vs Final Score คืออะไร?)	แก้ไขคำศัพท์ และอธิบายที่มาของข้อมูล (Test Cases): - เปลี่ยนคำว่า Baseline เป็น Initial Test (รันเทสรอบแรก) และเปรียบเทียบกับ Refined Test (รันรอบสองหลังแก้บั๊ก) เพื่อดูพัฒนาการ - ข้อมูลทดสอบไม่ได้สุม แต่เป็นการทำ Functional Test Cases (คลุมทั้งเคสปกติและ Edge Cases เพื่อตักคนโกงกติกา)
การให้คะแนนดูจับต้องไม่ได้ (Subjective): (ต้องมีสมการหรือตัวเลขที่ชัดเจน)	แบ่งการประเมินเป็น 2 ระยะ (Quantitative Metrics): 1. Functional Test (ทดสอบฟังก์ชัน): ตรวจสอบด้วยสคริปต์อัตโนมัติแบบ Pass/Fail (Exact Match) 2. Pilot Test (ทดสอบประสบการณ์): ให้ผู้เล่น 3 กลุ่ม (Casual, Veteran, Tactician) ประเมินด้วยคะแนน 1-5 ตามมาตรฐานงานวิจัย
ขาดการวิเคราะห์ข้อผิดพลาด (Error Analysis):	เพิ่มหัวข้อวิเคราะห์สาเหตุ (Root Cause Analysis)

System Overview



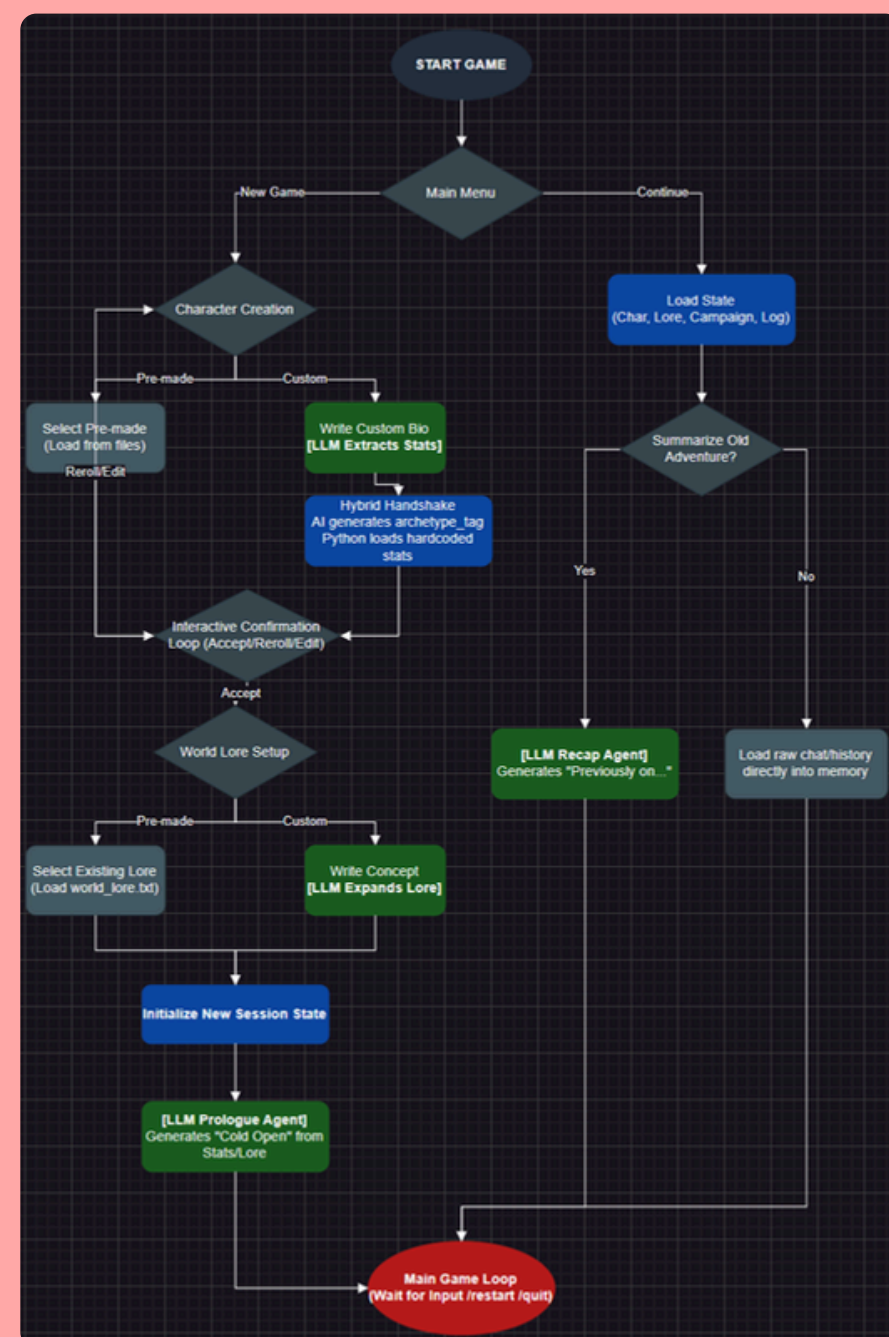
DEMO



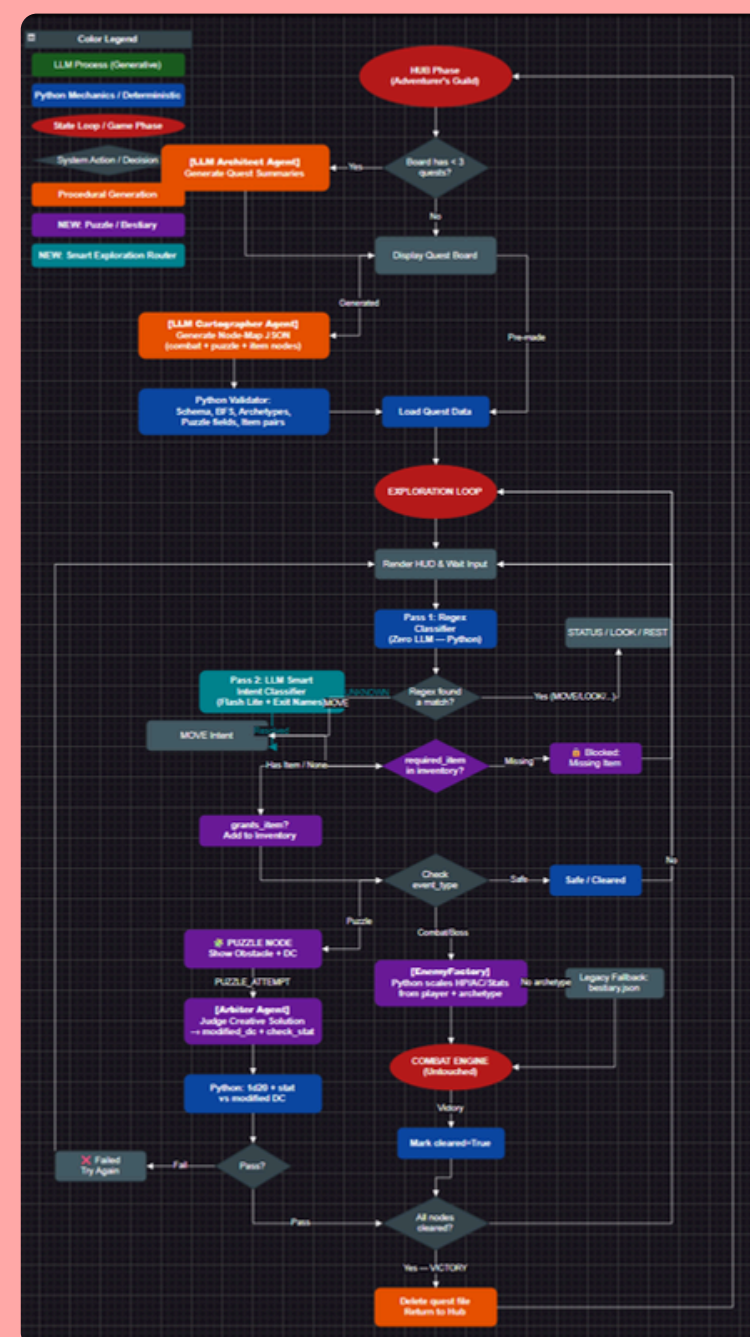
DEMO

VDO WITH TEXT PHRASE EXPLAIN

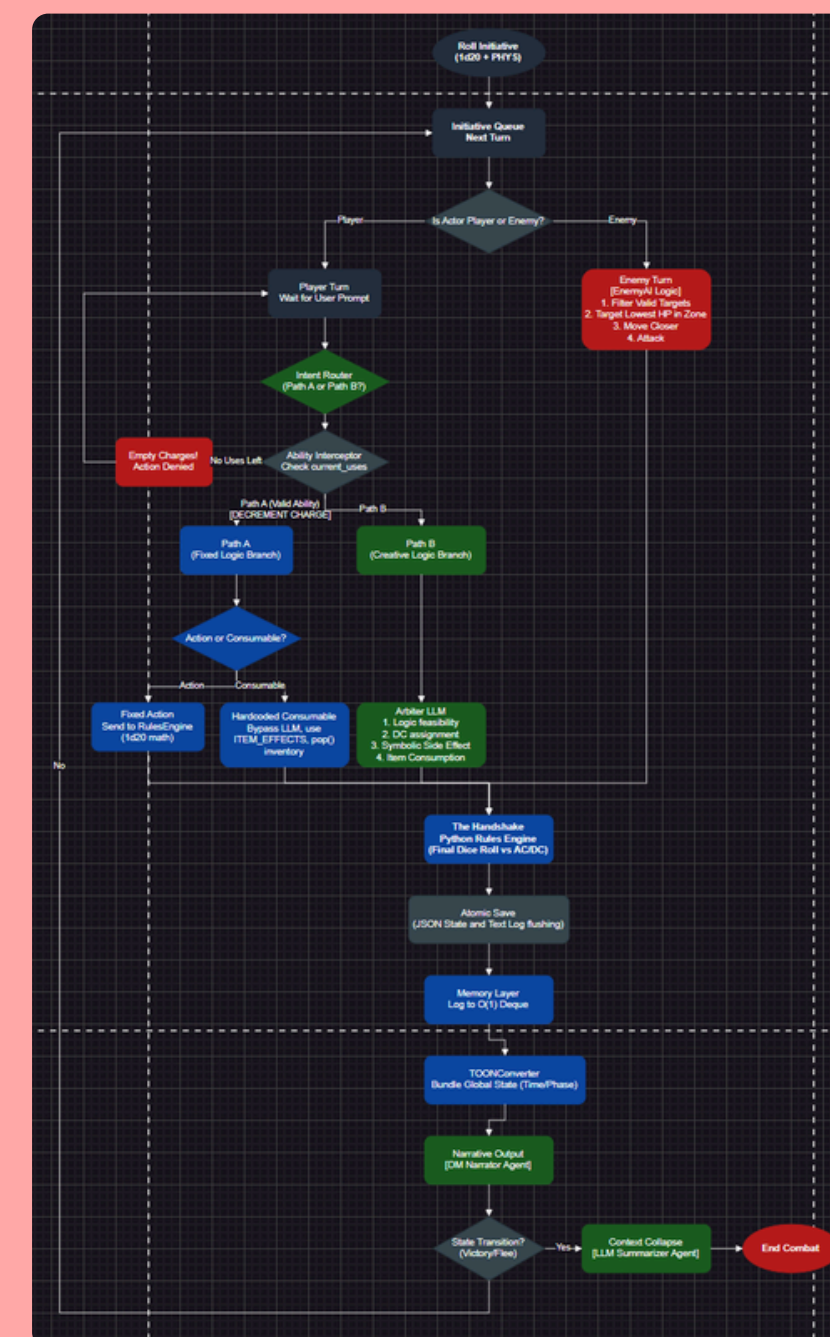
ภาพรวมสถาปัตยกรรมระบบ



START MENU

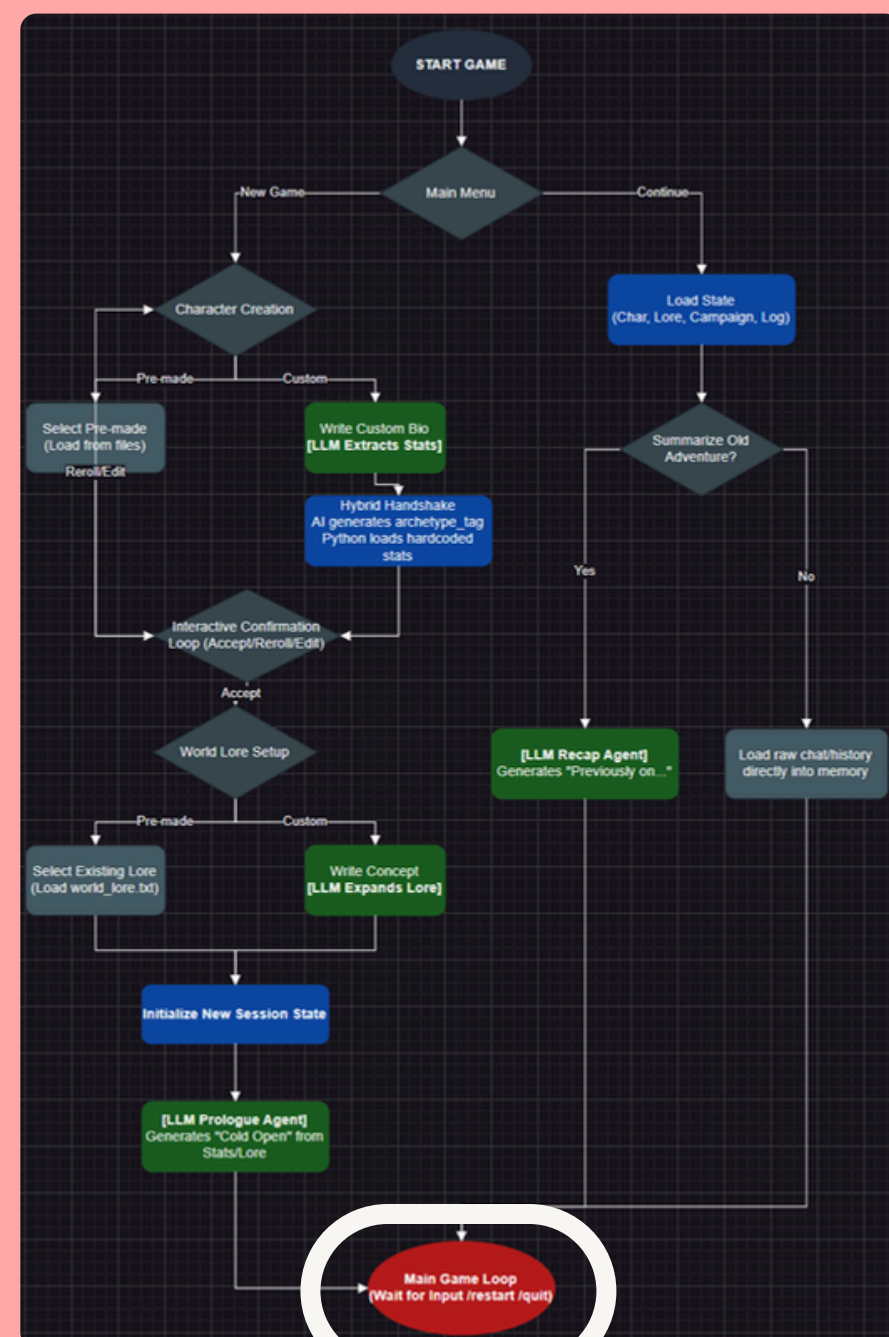


EXPLORATION MENU

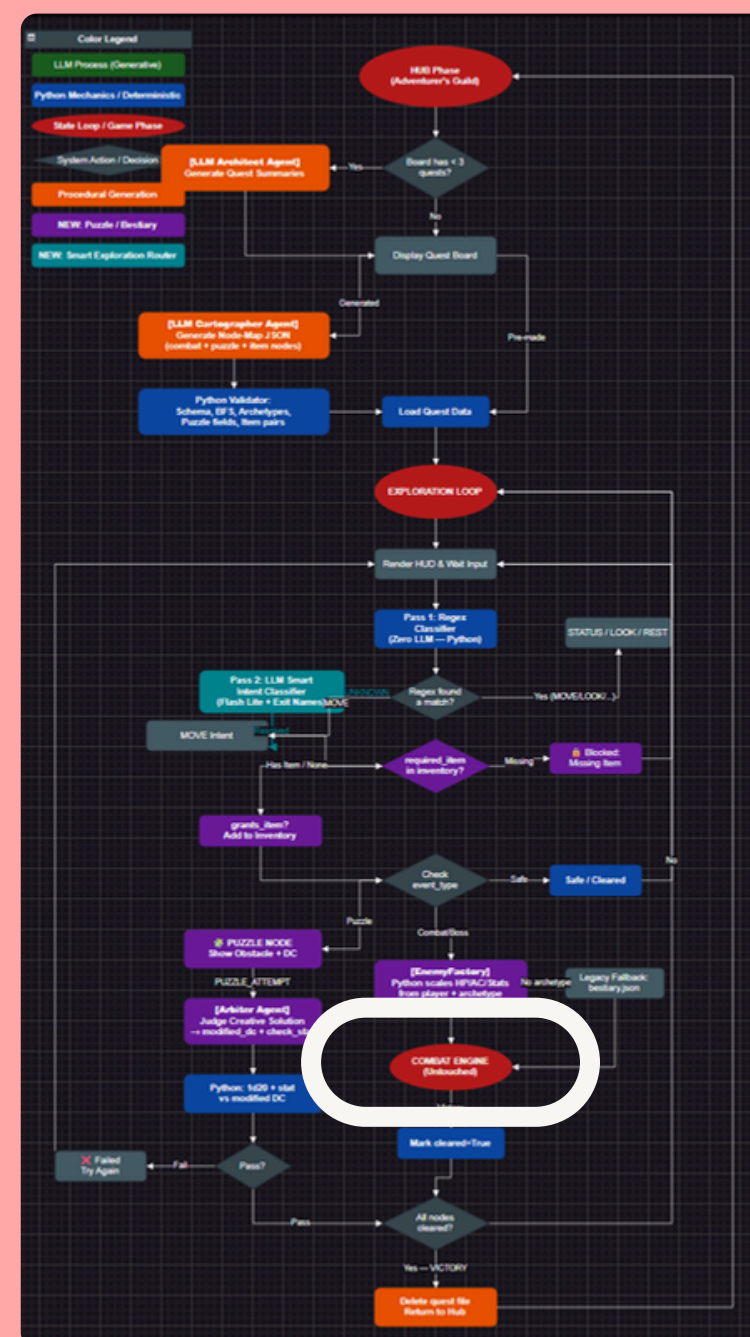


COMBAT MENU

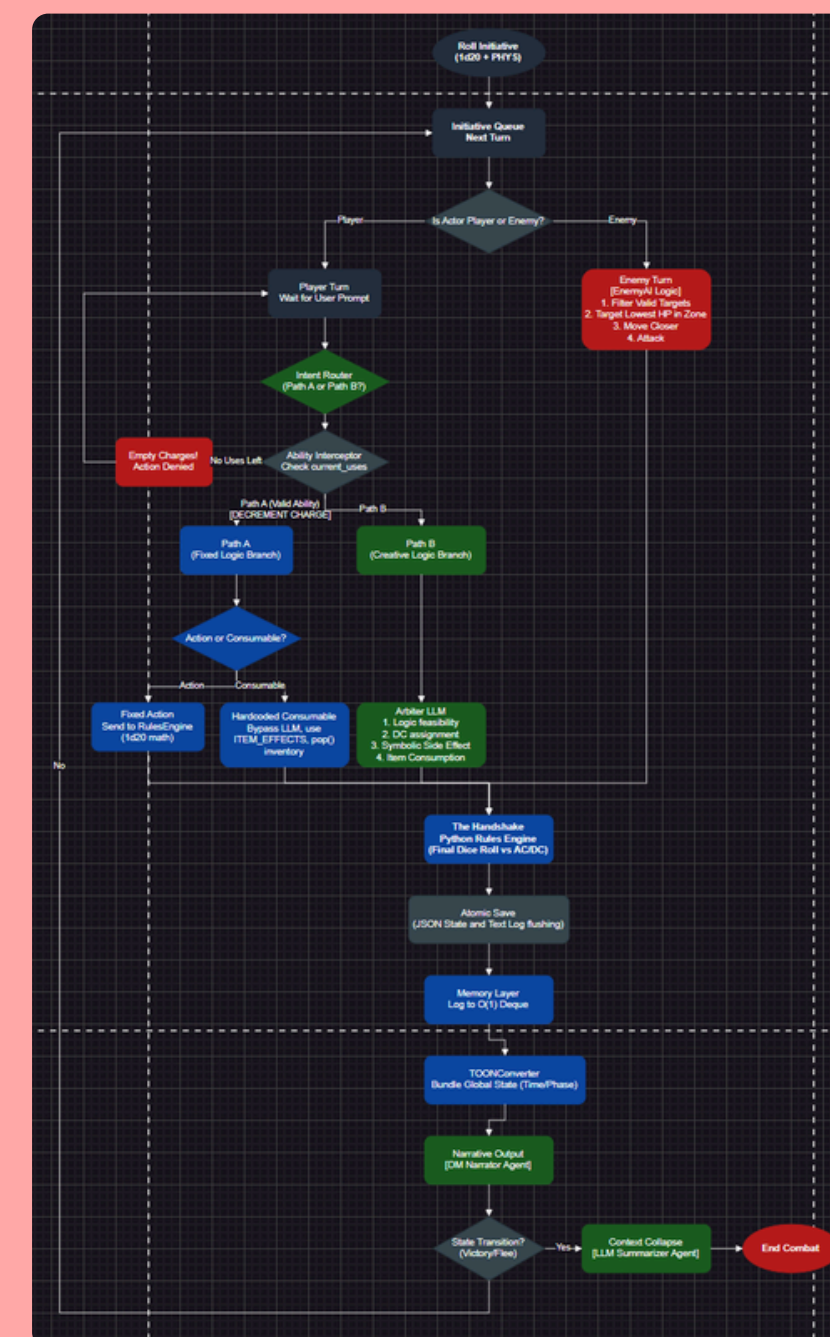
ภาพรวมสถาปัตยกรรมระบบ



START MENU

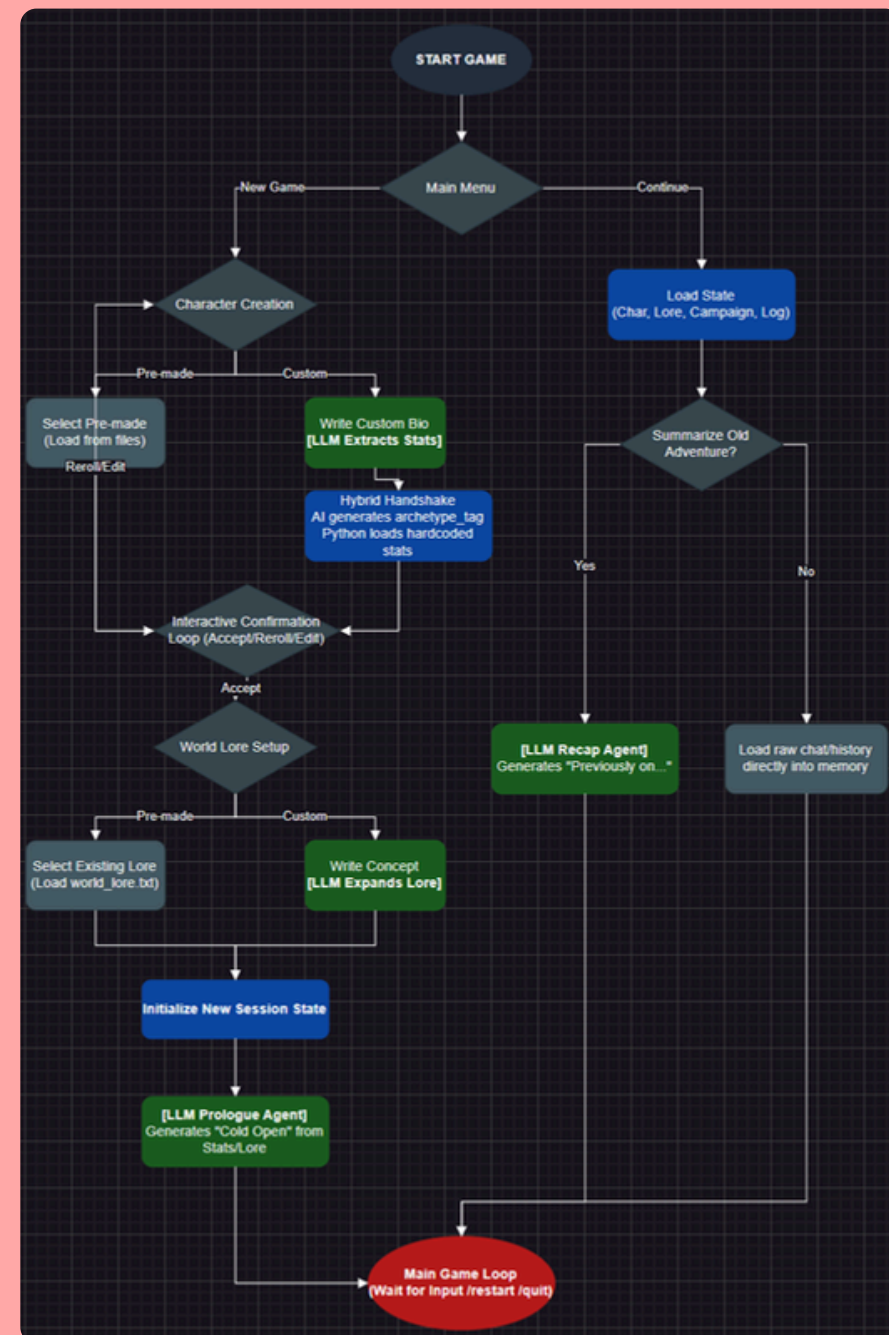


EXPLORATION MENU

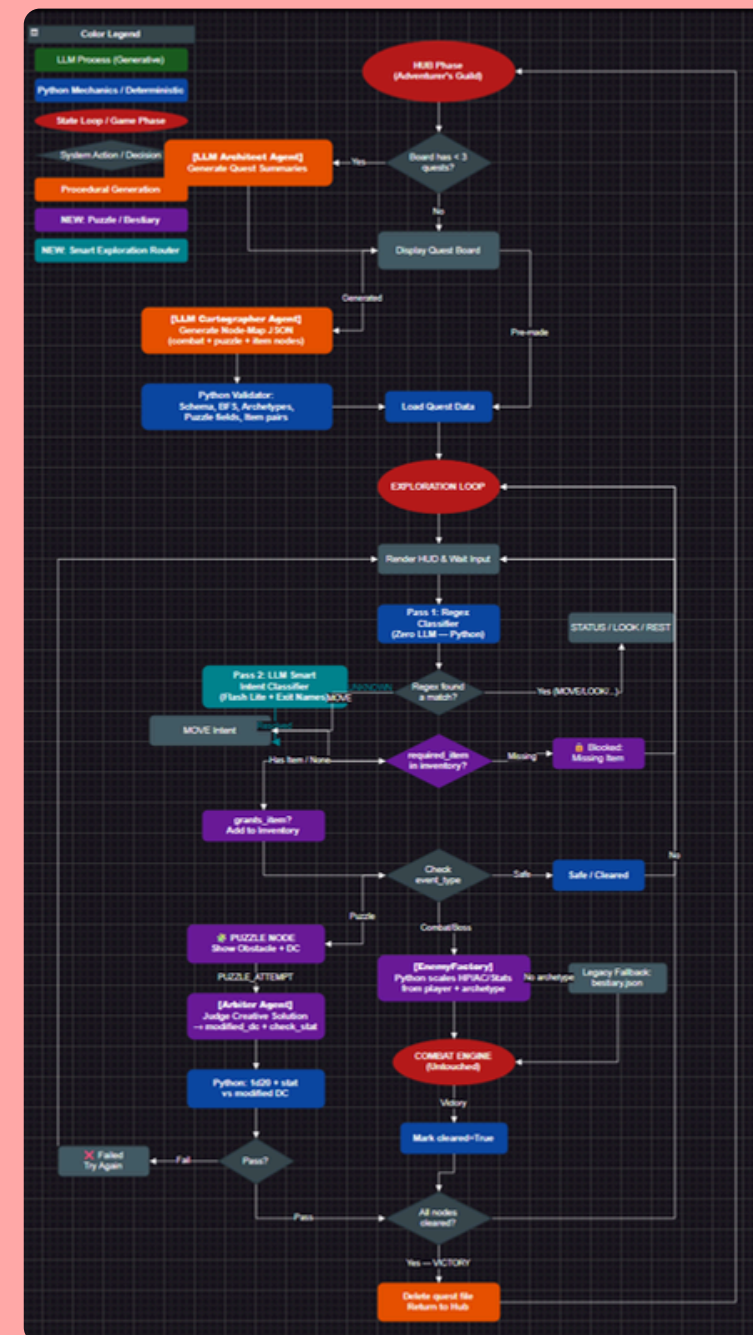


COMBAT MENU

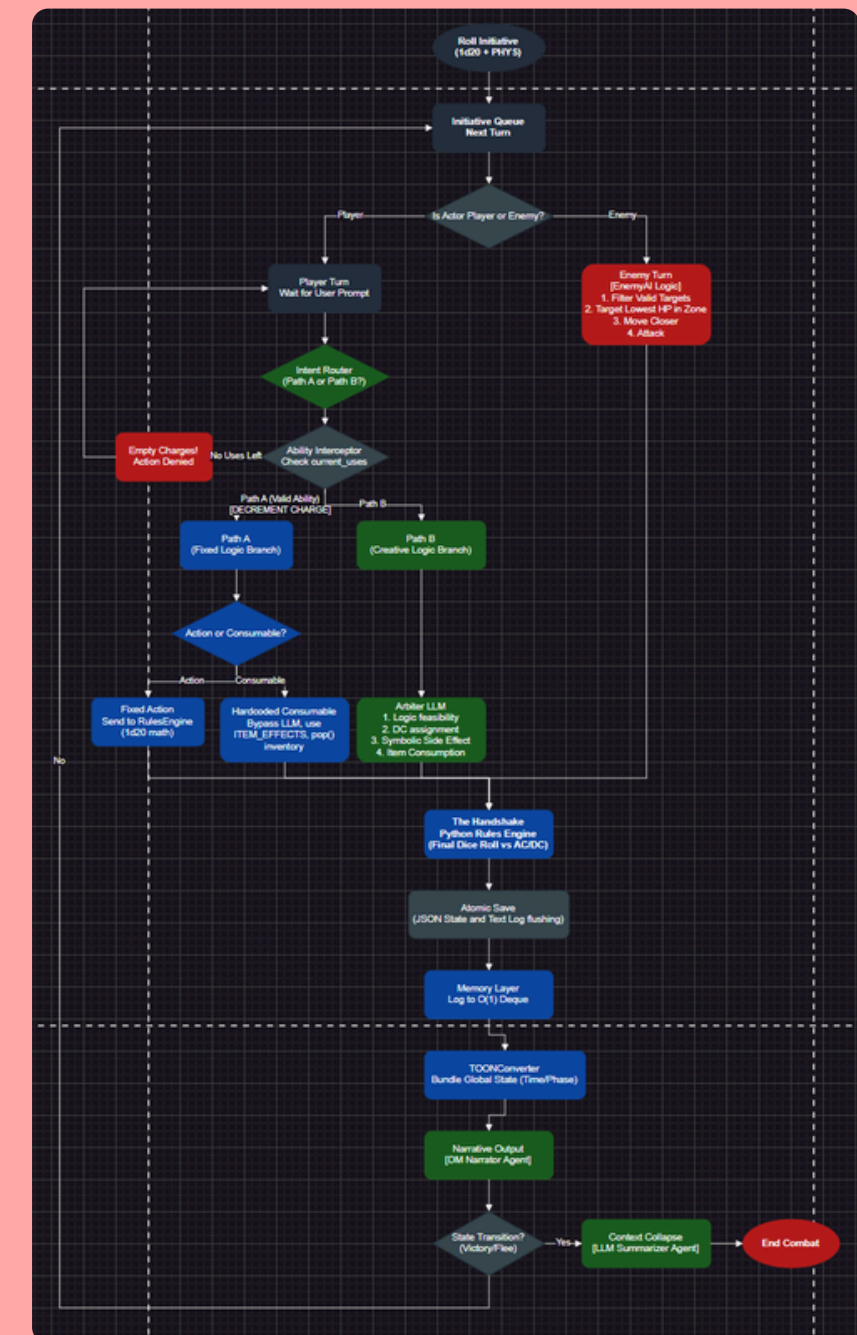
ภาพรวมสถาปัตยกรรมระบบ



START MENU

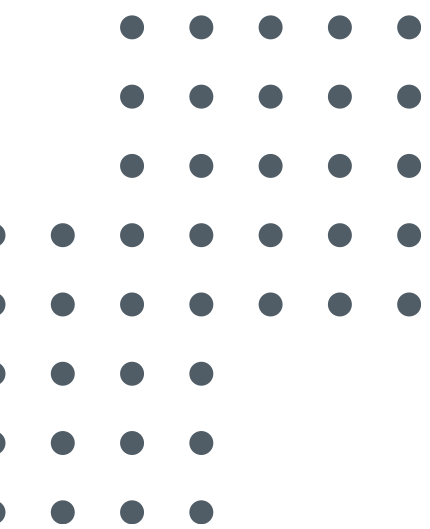


EXPLORATION MENU



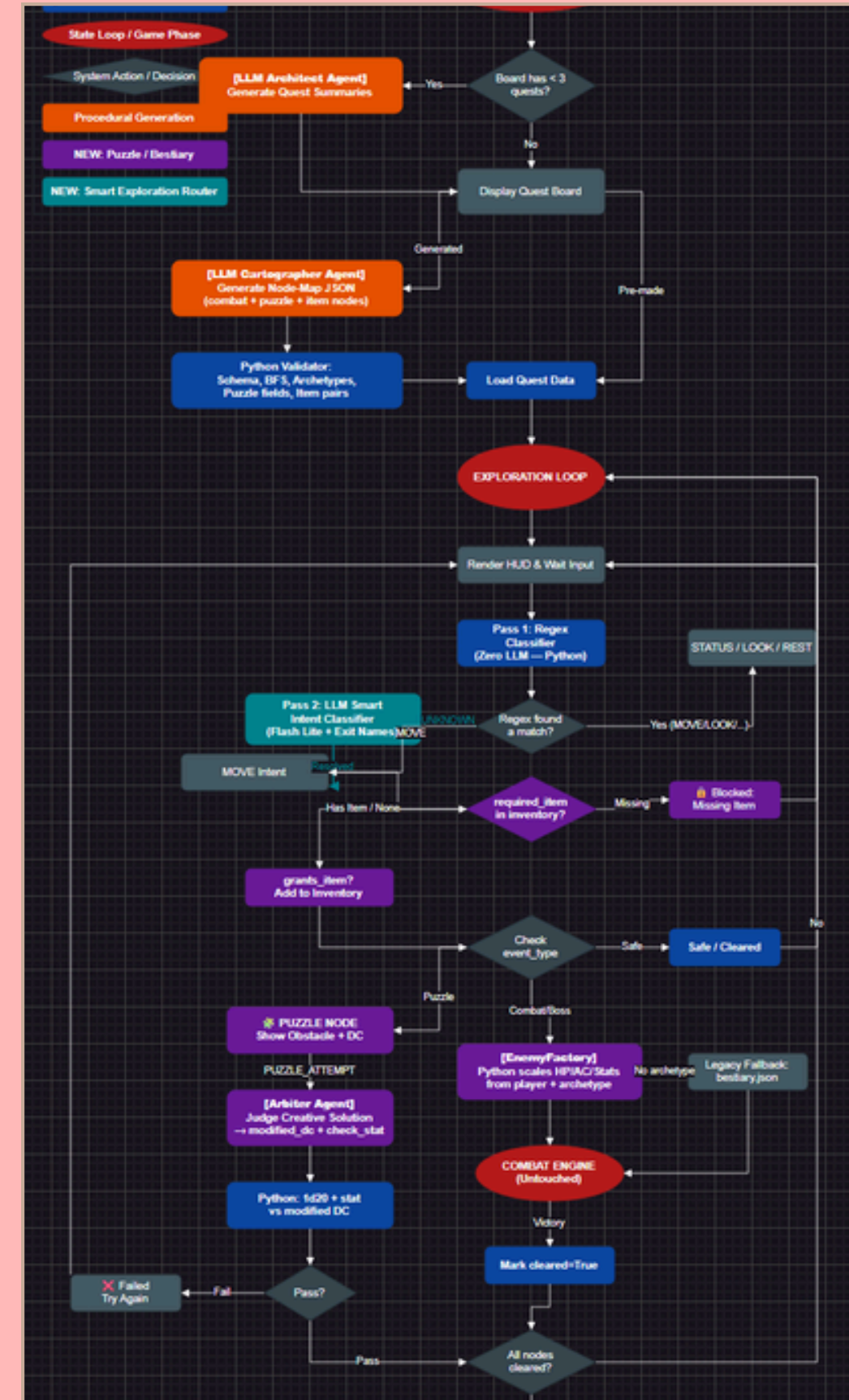
COMBAT MENU

Implementation & Progress Update



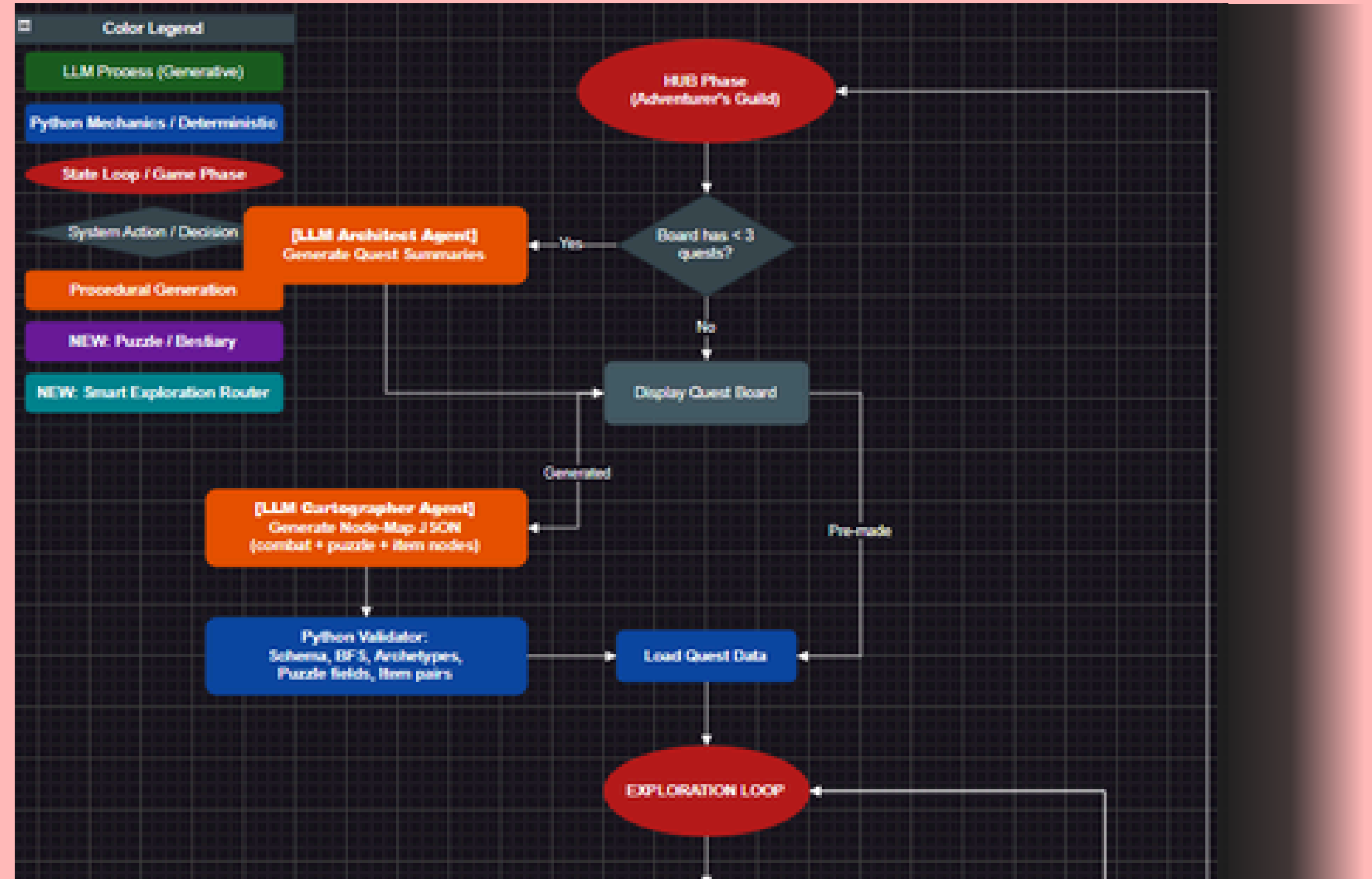
The Engine Flowchart

Exploration



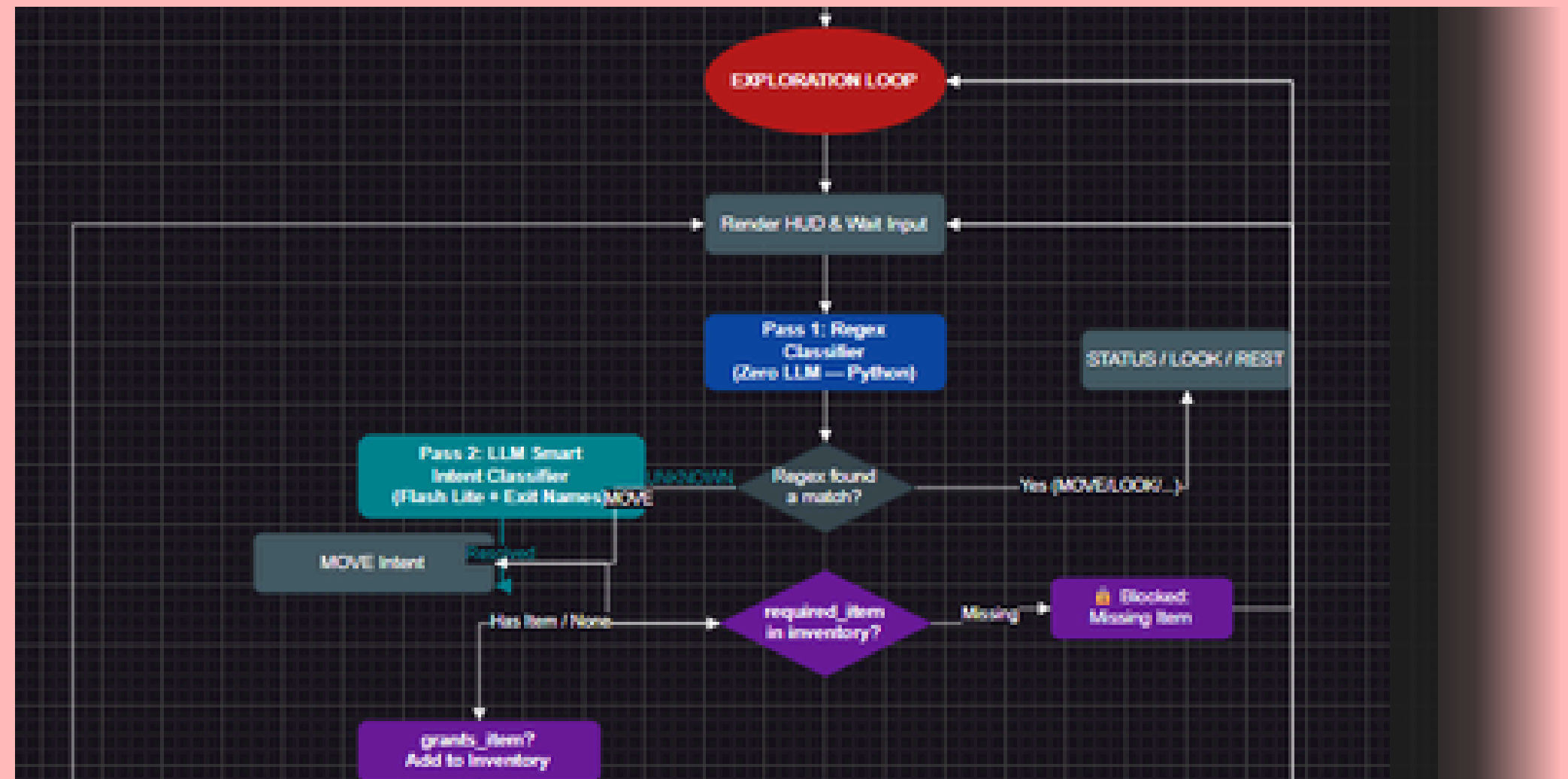
The Engine Flowchart

Exploration



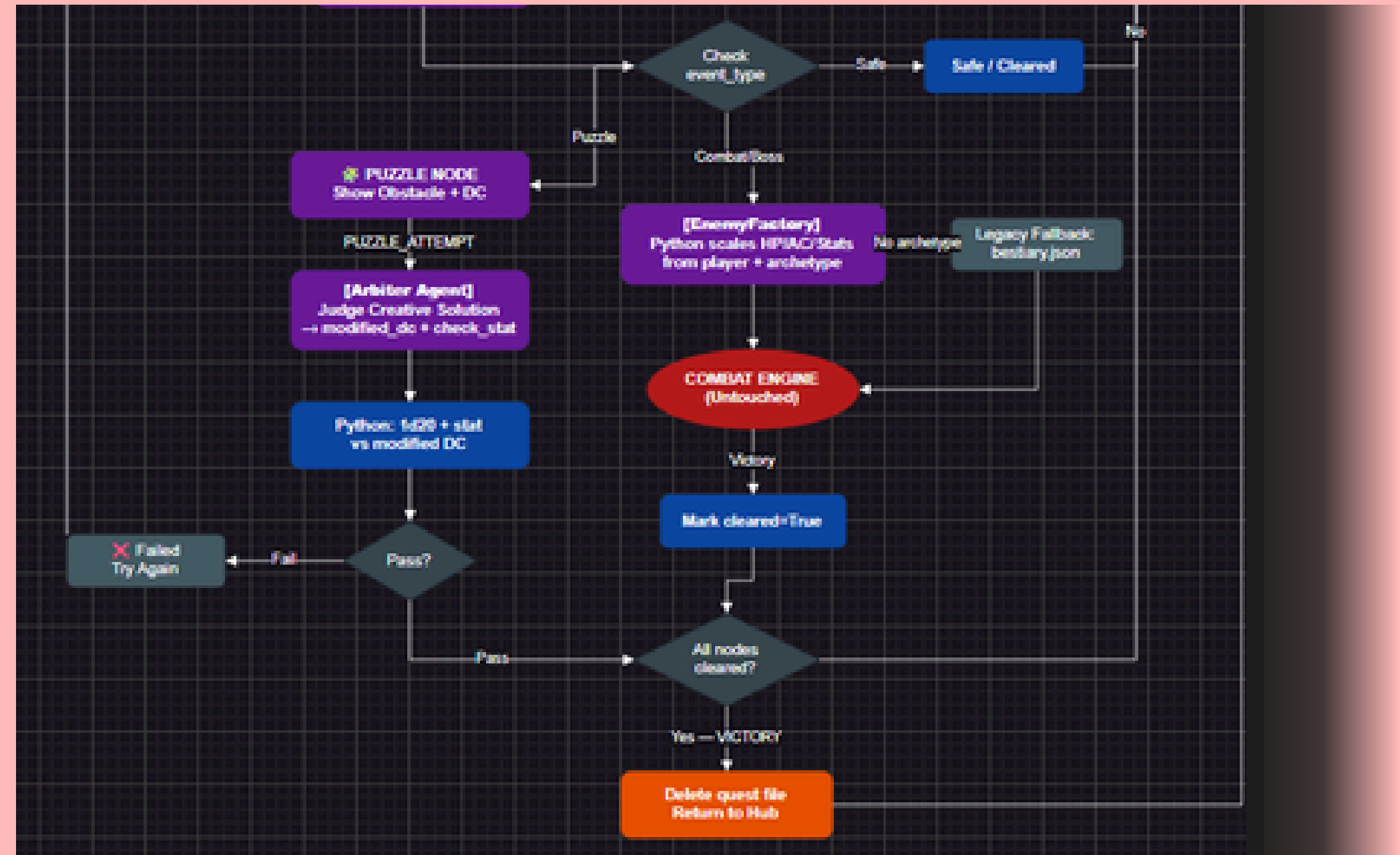
The Engine Flowchart

Exploration



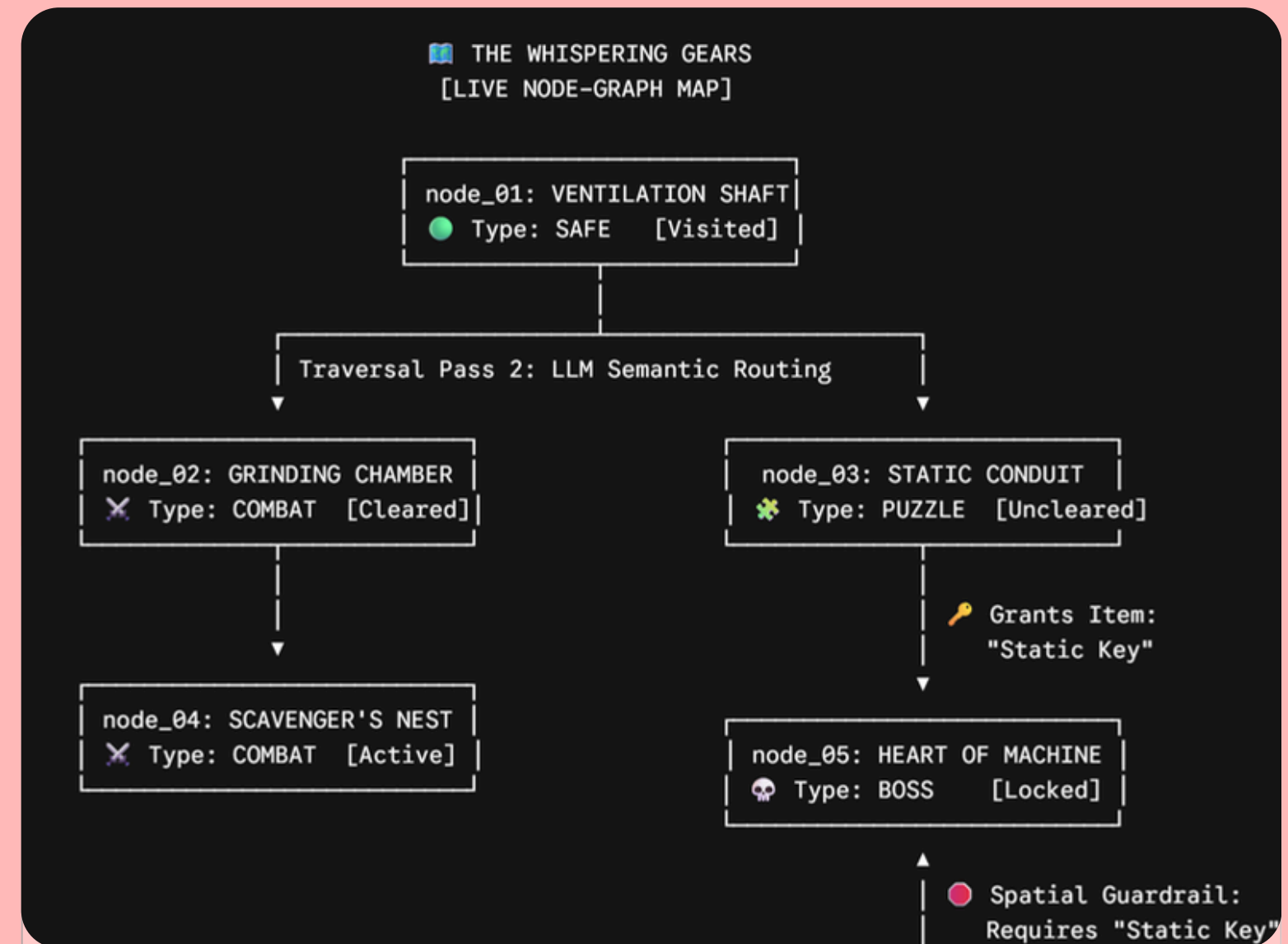
The Engine Flowchart

Exploration



THE NODE-BASED POINT-CRAWL SYSTEM

- Gridless Navigation: เปลี่ยนแบบตาราง (Grid) เปลี่ยนเป็นการเดิน Node-to-Node ที่เข้ากับการเล่าเรื่องของ AI มากกว่า
- Zero-Token Spatial Guardrail: ป้องกัน AI Hallucination ด้วยการให้ Python ในการการเดิน ทุกห้องถูกจำกัดเส้นทางด้วย Array ของ Python (เช่น ห้อง A ไปได้แค่ B และ C)
- Gating: ใช้ Python คำนวณเงื่อนไขการผ่านประตู (Key/Lock) โดยไม่ใช้ LLM



ใส่โลโก้ Python คงคู่กับโลโก้ Gemini

THE HYBRID SEMANTIC EXPLORATION ROUTER

แปลงข้อความ Roleplay อีสระ ให้กลายเป็นคำสั่ง

Pass 1: Regex Classifier (Python)

Zero-Cost & Ultra-Fast: ดักจับคำสั่งพื้นฐาน (เช่น *look, status, inventory*)
Benefit: ประหยัดค่า *Token API* สำหรับคำสั่งซ้ำซาก

Pass 2: LLM Smart Intent (Gemini Flash Lite)

Fallback Resolution: ทำงานเฉพาะเมื่อผู้เล่นพิมพ์ประโยคซับซ้อน (เช่น "ค่อยๆ ไต่เชือกลงไปห้องใต้ดิน")
Contextual Matching: แปลงคำพูดเป็นเจตนาโดยใช้ *LLM*

DUAL-AGENT QUESTS (ARCHITECT & CARTOGRAPHER)

The Quest Cartographer (นักเขียนแผนที่)

```
{  
  "node_id": "node_04",  
  "connected_to": ["node_02", "node_05"],  
  "event_type": "combat",  
  "cleared": false,  
  "enemy_tag": "goblin_ambusher"  
}
```

ตัวอย่างใน 1 Node ที่สร้าง

ใช้งานเฉพาะเมื่อผู้เล่นรับภารกิจเฉพาะ
เท่านั้น ใช้ออกแบบโครงสร้างหรือดันเจี้ยน

สร้างข้อมูล Toon แปลงเป็น JSON :
Node เส้นทางที่เชื่อมต่อ กับดัก และตำแหน่งเกิดศัตรู

DUAL-AGENT QUESTS (ARCHITECT & CARTOGRAPHER)

The Quest Architect (นักเล่าเรื่อง)

```
{
  "node_id": "node_02",
  "name": "Storage Room",
  "connected_to": ["node_01", "node_03"],
  "event_type": "combat",
  "enemy_tag": "goblin",
  "visited": false,
  "cleared": false,
  "lore_fragment": "### The Infestation\nThe scratch marks..."
}
```

สร้าง “Pitch” สำหรับหน้า Quest Board

สร้าง Title, Lore Fragment, Objective

ไม่ยุ่งกับคณิตศาสตร์ แค่สร้าง Hook ให้คนสนใจ
อิงจากเนื้อเรื่องโลกเดิม

DUAL-AGENT QUESTS (ARCHITECT & CARTOGRAPHER)

The Quest Architect (นักเล่าเรื่อง)

```
{  
  "node_id": "node_02",  
  "name": "Storage Room",  
  "connected_to": ["node_01", "node_03"],  
  "event_type": "combat",  
  "enemy_tag": "goblin",  
  "visited": false,  
  "cleared": false,  
  "lore_fragment": "### The Infestation\nThe scratch marks..."  
}
```

สร้าง “Pitch” สำหรับหน้า Quest Board

สร้าง Title, Lore Fragment, Objective

ไม่ยุ่งกับคณิตศาสตร์ แค่สร้าง Hook ให้คนสนใจ
อิงจากเนื้อเรื่องโลกเดิม

DYNAMIC CONTENT & SYSTEM SAFETY ARCHITECTURE

7-Layer JSON Guardrail

- ระบบ *Python* ตรวจสอบความสมบูรณ์ของ *JSON Map* ก่อนให้ผู้เล่นเล่นเสมอ (หากพัง 2 ครั้ง จะถึง *Fallback Map*)
- *Key Validations*:
- ตรวจสอบโครงสร้างพื้นฐาน (*Top-Level & Required Fields*)
- บล็อกห้องที่ไม่มีอยู่จริง (*Dangling Reference*) และคุณสมบัติจุดเกิดศัตรู
- *BFS Algorithm*: คำนวณกราฟเพื่อพิสูจน์ว่าทุกห้องเดินเชื่อมถึงกันได้จริง

"Skeleton & Flesh" Dynamic Bestiary

- *The Flesh (LLM)*: คิดเฉพาะชื่อมอนสเตอร์และคำบรรยายท่าโจมตี (*Flavor Text*)
- *The Skeleton (Python)*: คำนวณตัวเลขสถานะ ดาเมจ และจำนวนเลือด (*HP*) 100%
- *Dynamic HP Math*: $\text{Enemy HP} = \text{Player Max HP} \times \text{Archetype Multiplier}$
- *Minion* (x0.4), *Skirmisher* (x0.7), *Brute* (x1.2), *Boss* (x2.5)

อัฟเตอร์ระบบ COMBAT และกติกาที่เข้มงวดขึ้น

HARDCODED PYTHON ENFORCEMENT

- ใช้โค้ด Python บล็อกกติกาโดยให้ผู้เล่นไว้ที่ 1 Move + 1 Major Action ต่อหนึ่งเทิร์น
- หากพยายามทำหลายแอ็กชันซ้ำซ้อนกัน (เช่น พิมพ์สั่งโจมตี 3 ครั้งติดกัน) ระบบจะดักจับและส่งคำสั่งเข้า Path Creative เพื่อทำการ DENY ทันที
- สร้างระบบสกิลเฉพาะ Class โดย คำนวณใน Python 100%

การใช้ LORE ผู้เล่นปรับค่าความยาก

- ระบบจะนำข้อมูล Player Lore ของผู้เล่น ส่งให้ Arbiter Agent นำไปวิเคราะห์ร่วมกับคำสั่งที่พิมพ์เข้ามา
- หากแอ็กชันที่ทำสอดคล้องกับประวัติของตัวละคร ระบบจะลดค่าความยากลง 3 ถึง 5 แต้ม ทันที ทำให้ระบบรองรับการ Roleplay ได้มากขึ้น

ระบบ DYNAMIC PUZZLES

- ขยาย Arbiter Agent มาใช้ควบคู่กับ Puzzle Nodes เพื่อเปิดโอกาสให้ผู้เล่นพิมพ์ไอดีเดียวแก้ Puzzle ด้วยภาษาพูดได้อย่างอิสระ
- LLM ทำหน้าที่คัดกรองความสร้างสรรค์ของไอดีเดียว จากนั้นแปลงผลกลับมาเป็นค่าความยาก เพื่อให้ Python ทำการทอยเต๋า 1d20 ดำเนินเกมต่อ

แอปเดทภาพรวมระบบ

ระบบบันทึกข้อมูล

- Real-time Traceability: บันทึกทุกลำดับการทำงาน อย่างละเอียด ทำให้สามารถตรวจสอบและ Resume เกมได้ทันทีจากจุดที่ออก

ระบบจำลองเวลาและคูลดาว์นสกีล

- Day/Night Cycle: โค้ด Python มีตัวแปรเวลา เพื่อติดตามเวลากลางวัน-กลางคืนโดยอัตโนมัติ และป้อนข้อมูลนี้ให้ AI นำไปใช้อ่านสถานะเพื่อบรรยายฉาก
- Strategic Time Chunks: เวลาของโลกเกมจะขยับขับเคลื่อนแบบก้าวกระโดด (Chunks) ผ่านการตัดสินใจพักผ่อนของผู้เล่นเท่านั้น:
 - Short Rest (พักสั้น): เร่งเวลาข้ามไป +1 ชั่วโมง ทันที
 - Long Rest (พักยาว): เร่งเวลาข้ามไป +8 ชั่วโมง ทันที
- Cooldown Math: ใช้ Python คำนวณระบบพักผ่อน (Short/Long Rest) เพื่อรีเซ็ตและห้กลับโควตาการใช้สกีลจริงตามกลไกเวลา

การแก้ไขสถาปัตยกรรม หลังจากการทดสอบ ALPHA PILOT TEST



From MS-DOS to Natural Language Exploration

- *Before* (ระบบเก่า): เวลาผู้เล่นจะเดินข้ามห้องในโหมด *Exploration* ต้องพิมพ์คำสั่งแบบ *Command-line* ที่อ้อๆ (เช่น *move roomname* หรือ *rest*) ซึ่งทำลายความ *Immersive* และการสวมบทบาท (*Roleplay*) โดยสิ้นเชิง
- *The Upgrade*: เราได้ทำการขยายสโคปของ *Intent Router* (ที่แต่เดิมใช้แค่ในระบบต่อสู้) ให้ครอบคลุมมาถึงระบบสำรวจ (*Exploration*) ด้วย
- *After* (ระบบปัจจุบัน): ผู้เล่นสามารถพิมพ์ภาษาธรรมชาติได้เลย เช่น "ฉันเหนื่อยแล้วค่อยๆ เดินย่องเข้าไปในห้องสมุด" *Intent Router* จะทำการสกัดเจตนา (*Extract Intent*) เปลี่ยนเป็นคำสั่ง *MOVE* และรับหลังบ้านให้โดยอัตโนมัติ ทำให้ผู้เล่นรู้สึกเหมือนกำลังคุยกับมมนุษย์ (*Dungeon Master*) จริงๆ 100%



Narrator Node ID Sanitization

- *The Bug*: เวลาผู้เล่นเดินเข้าห้องใหม่ LLM นำหมายเลข *Node* มาเล่าให้ผู้เล่นฟัง ทำลายความ *Immersive* ของเกม
- *The Fix*: ทำการสร้าง *Layer* แยกจาก *Node* เป็นชื่อห้องก่อนส่งไปให้ LLM บรรยาย



Context-Aware Narration

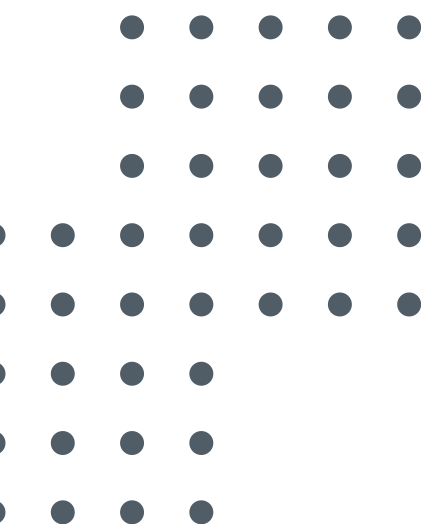
- *The Bug*: ผู้เล่นร้ายเวทย์แก้ *Puzzle* สำเร็จ แต่ LLM เล่าจากด้วยความโหดร้าย เพราะเอนจินส่งคำสั่งใช้เกมเพลตเชื่อมโยง "*Room Cleared*" บล็อกเดียวกันกับโหมด *combat*
- *The Fix*: ทำการแยก *combat* กับ *puzzle* โดยเพิ่มสถานะ *cleared_by* หากตัวแปรเป็น "*puzzle*" ระบบจะไม่บรรยายเกี่ยวกับศัตรู



Self-Targeting Guardrails

- *The Bug*: เมื่อผู้เล่นดื่มยาเพิ่มเลือดเพื่อรักษาแผลตัวเอง ระบบ *Intent Router* ถอดเจตนาเล็งเป้าหมายส่งกลับมาเป็นตัวเอง แต่ระบบ *Targeting* ดักเช็คและวิ่งหาแค่รายชื่อ ศัตรู จึงเกิน *Error*
- *The Fix*: เพิ่มระบบการ *Target* ใส่ตัวเองเข้าไป

System Evaluation & Pilot Study

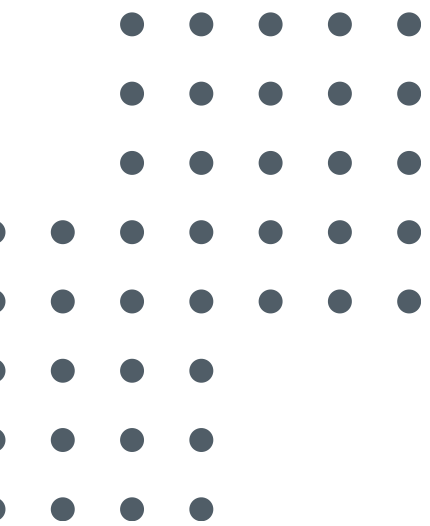


ตารางรวมการทดลอง

โครงสร้างการแบ่งขอบเขตการทวนสอบฟังก์ชันระบบและการประเมินผลโดยมนุษย์

1: Automated Functional Verification (ฝั่งซ้าย)	2: Human Validation & Pilot Study (ฝั่งขวา)
กลุ่มเป้าหมาย: สคริปต์ทดสอบระบบอัตโนมัติ 100%	กลุ่มเป้าหมาย: มนุษย์ผู้เล่นจริง (3 Gamer Personas)
ขนาดคำ: 90 Distinct Scenarios (8 หมวดหมู่หลัก)	ขนาดคำ: Closed Alpha Test + Final Pilot Test
- วัตถุประสงค์หลัก: - ตรวจสอบความถูกต้องทางคณิตศาสตร์และ State Machine ของเอนจิน Python - ทดสอบความเหนียวแน่นของระบบเลนกัน (Guardrails) ป้องกันคนโกงเกม	- วัตถุประสงค์หลัก: - ประเมินความยืดหยุ่นในการรับภาษาธรรมชาติที่ไม่ล็อกคำสั่ง - วัดค่าความคงเส้นคงวาและอารมณ์ร่วมของการเล่าเรื่อง (Immersion)
เครื่องมือชี้วัด: 4 Core Metrics (A_route, P_ground, S_sync, C_narr)	เครื่องมือชี้วัด: 5-Point Likert Scale (อ้างอิงงานวิจัย)

FUNCTIONAL VERIFICATION



โครงสร้างและการออกแบบชุดทดสอบ

90 สถานการณ์ ครอบคลุม 8 หมวดหมู่หลัก (รวม 24 กลไกย่อย)

Category A

Basic Combat & Rules

(19 Scenarios)

โจมตีปกติ, คำนวณ

Damage/AC, ระบบแพ้ทาง

Category B

Creative Actions

(11 Scenarios)

การกระทำอิสระนอกกฎ, ใช้สิ่ง

แวดล้อม

Category C

Inventory & Usage

(13 Scenarios)

การใช้ *Potion*, การเช็คของใน

กระเป๋า

Category D

Navigation & State

(14 Scenarios)

การย้ายห้อง, สภาพแวดล้อม

Category E

Multi-Target

(6 Scenarios)

การโจมตีหมู่ (AOE)

Category F

The "Yes Man" Exploit

Guardrails (9 Scenarios)

ดักจับ *LLM Hallucination* (เสกข

อง, สร้างพลังเอง)

Category G

Context Bleed

(10 Scenarios)

ป้องกันข้อมูลผู้เล่นสลับกันในระบบ

Multi-Agent

Category H

Class Abilities & Combos

(8 Scenarios)

สกิลติดตัว, การหลบหนี, การทำ

Combo

โครงสร้างและการออกแบบชุดทดสอบ

INFOGRAPHIC PIE CHART

Normal Actions (คำสั่งกฎตายตัว)



- เป้าหมาย: ทดสอบความแม่นยำทางคณิตศาสตร์ของ Python Rules Engine (Mathematical Accuracy)
- ตัวอย่าง: "ฉันใช้ดาบฟันกอบลิน", "เดินไปห้องโถงทิศเหนือ"
- สิ่งที่ระบบต้องทำ: คำนวณทอยลูกเต๋า (Dice Roll) ปะทะ Armor Class, อัปเดต HP อย่างแม่นยำ 100%

Creative Actions (คำสั่งอิสระแบบ Roleplay)



- เป้าหมาย: ทดสอบตรรกะและการประเมินความยาก (Difficulty Class - DC) ของ LLM Arbiter
- ตัวอย่าง: "ฉันกวาดทรายบนพื้นป่าใส่ตากอบลินให้มันมองไม่เห็น"
- สิ่งที่ระบบต้องทำ: LLM ต้องคำนวณ DC ที่เหมาะสม (เช่น สั่งให้ทอย DEX) และให้ผลลัพธ์เป็นสถานะ Stunned หรือ Blinded ได้โดยไม่ต้องมีกฎ Hardcode

System Abuse / Edge Cases (การจงใจแหกกฎกติกา)



- เป้าหมาย: ทดสอบความแข็งแกร่งของ Python Guardrails ในการสกัดกั้นการโกงเกม
- ตัวอย่าง: "ฉันดื่มน้ำพิษรักษาแผล" (ทั้งที่ไม่มีพิษในกระป๋อง), หรือ "ฉันเดินหนี ฟันกอบลิน แล้วฮิลตัวเอง" (ทำ 3 แอคชั่นใน 1 เทิร์น)
- สิ่งที่ระบบต้องทำ: ระบบต้องปฏิเสธคำสั่ง (Reject) และแจ้งเตือนผู้เล่น โดยไม่ยอมให้ State ของเกมเปลี่ยน

การประเมินผลใช้ตัวชี้วัด 4 ด้าน

State Synchronization (S_sync)

ความหมาย: ความแม่นยำในการคำนวณและอัปเดตสถานะเกม (Mathematical & State Accuracy)

เกณฑ์การผ่าน: HP ลดตรงเป๊ะตามลูกเต๋าทิ้ง, สถานะตัวละครอัปเดตถูกต้อง 100% ห้ามมีข้อผิดพลาดทางตัวเลข

Routing Accuracy (A_route)

ความหมาย: ความแม่นยำในการแยกแยะประเภทคำสั่ง (Intent Classification)

เกณฑ์การผ่าน: Intent Router ต้องแยกได้ว่าคำสั่งไหนเป็น กฎตายตัว (Fixed Action) คำสั่งไหนเป็น เปิดกว้างอิสระ (Creative Action) เพื่อส่งไปรันในระบบที่ถูกต้อง

Rule Grounding (P_ground)

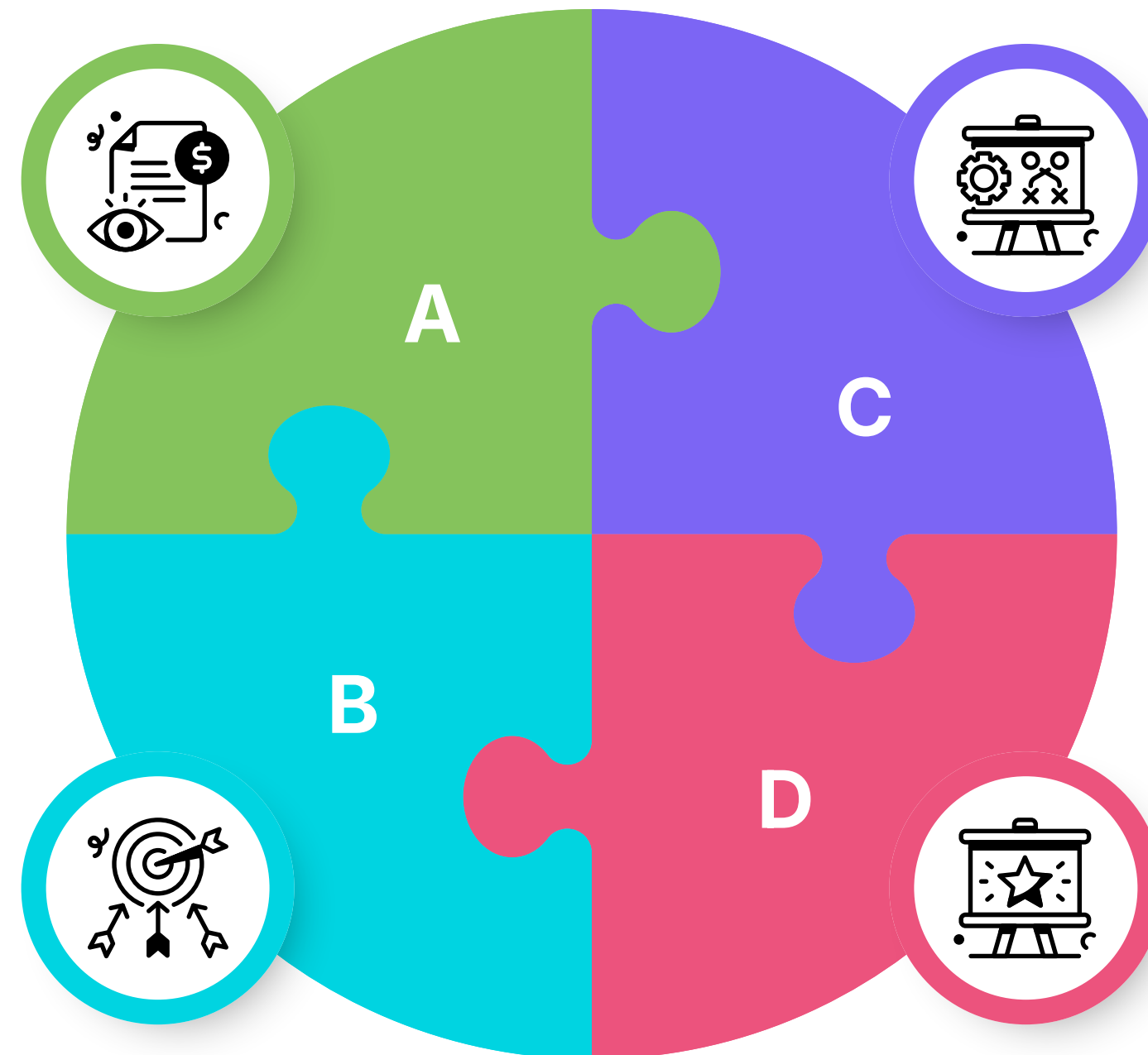
ความหมาย: ความสามารถในการยึดติดกับกฎและดักจับข้อผิดพลาด (Guardrail Enforcement)

เกณฑ์การผ่าน: ระบบต้องปฏิเสธคำสั่งที่ผิดกฎ (เช่น ใช้ไอเทมที่ไม่มี, ตีมอนสเตอร์ที่ตายแล้ว) โดยไม่ถูก LLM หลอก (No Hallucination)

Narrative Coherence (C_narr)

ความหมาย: ความสมจริงและสลับไหลของการบรรยาย (Narrative Quality)

เกณฑ์การผ่าน: เนื้อเรื่องที่ตอบกลับต้องสอดคล้องกับผลลัพธ์การคำนวณหลังบ้าน (เช่น ถ้าตีพลาด ระบบบรรยายต้องบอกว่าตีพลาด ไม่ใช่บอกว่าโดน)



โครงสร้างข้อมูลทดสอบ 1 Node

ตัวอย่างหน้าต่างของ Scenario ที่ป้อนเข้าระบบอัตโนมัติ ซึ่งจะกำหนด Input และ Expected Output ไว้อย่างชัดเจน

```
{
  "scenario_id": "C-42",
  "category": "Inventory & Guardrails",
  "player_input": "I drink a potion of invincibility.",
  "mock_state": {
    "inventory": ["iron_sword"] // ไม่มี potion ในกระเป๋า
  },
  "expected_outcome": {
    "expected_route": "FIXED_USE_ITEM",
    "is_success": false, // ระบบต้องปฏิเสธ (Reject)
    "state_changes": {
      "player_hp": "UNCHANGED" // HP ต้องไม่เปลี่ยน
    },
    "grounding_check": "MUST_FAIL_INVENTORY_CHECK"
  }
}
```

การทดสอบรอบแรกและการวิเคราะห์ข้อผิดพลาด

ในการรันชุดทดสอบ 90 Scenarios รอบแรกด้วยสถาปัตยกรรม 2 เส้นทาง (Two-Path Architecture) ก่อนการทำ Bug-Fixing ระบบ
ทำคะแนนได้ในระดับที่น่าพอใจ แต่ยังพบข้อผิดพลาดในเชิงระบบ (System-level defects)

พบปัญหา 3 กลุ่มหลักที่ต้องได้รับแก้ไข

- Total Scenarios: 90
- Total Passed: 79 (87.8% Pass Rate)
- Total Errors: 11 Failures (10 Critical, 1 Low)
- Failures Caught: นำมาทำ Error Analysis ซึ่ง
ส่วนใหญ่ เกี่ยวข้องกับด้าน Guardrails และ Engine
Sync Failure

1. Action Economy Guardrail Bypasses

ปัญหาที่พบ (ตัวอย่าง C-42, C-43): บอทจงใจสั่งคำสั่งผิดกฎ เช่น "ฉันทิ่มโพชั่น" (ทั้งที่
กระเป๋าว่าง) หรือ "เดินไปไกลๆ แล้วเดินกลับมาในเทิร์นเดียว" ตัว Python Rules
Engine ควรจะ Deny แต่ Guardrail กลับมีช่องโหว่ ปล่อยให้ is_success=True ผ่านไป
อัปเดต State หน้าตาเฉย

2. API Schema Violation & Crash

ปัญหาที่พบ (เคส G-71): ในการทดสอบ Live API ขอกำเนิดเคสต์สดๆ ระบบหลังบ้าน
เกิดแครชกะทันหัน (unexpected keyword argument 'model_name') เพราะตัว
Quest Cartographer สื่อสารกับคลาส BaseLLMProvider ผิดพลาด ทำให้โครงสร้าง
JSON แตก

3. Unimplemented Execution Stubs

ปัญหาที่พบ (เคส H-85, H-88): เมื่อบอททดสอบพยายามวิ่งหนีออกจากฉากร (Flee)
หรือทำคอมโบ (Move and Smite) ตัว Intent Router แยกแยะคำสั่งได้เป๊ะ แต่เมื่อส่ง
ไปรันที่ Rules Engine กลับเกิด State Sync Failure เพราะฟังก์ชันยังไม่รองรับ
สถานการณ์นี้

ผลลัพธ์การทดสอบรอบสุดท้าย

หลังจากแก้ไขส่วนใหญ่ เราย่นำระบบกลับมารันชุดทดสอบ 90 Scenarios อีกครั้ง

- Routing Accuracy (A_route): 100.0%
(แยกแยะประเภทคำสั่งได้ถูกต้องทั้งหมด)
- Grounding Precision (P_ground):
100.0% (ดักจับการพิมพ์โก่งเกม และ
บังคับใช้กฎได้สมบูรณ์แบบ)
- Narrative Coherence (C_narr): 100.0%
(บรรยายเนื้อเรื่องได้สั่นไหว ไม่หลุดคีย์
ตัวแปร)
- State Synchronization (S_sync): 94.4%
(คณิตศาสตร์หลังบ้านและการอัปเดต
State สำเร็จ 85 จาก 90 เคส)

Error Analysis

- Routing Ambiguity (1 เคส): ระบบยังสับสนเวลาสั่ง "ใช้ไอเทมที่ไม่มีในกระเป๋า"
โดยเผลอส่งคำสั่งไปหมวด Creative (แต่งนิยาย) แทนที่จะเตะตกทันที
- LLM Grounding Anomaly (1 เคส): ตัวจัดหมวดหมู่ไอเทม (Item Categorizer)
ทำงานผิดพลาดเมื่อเจอคำศัพท์กำกวม
- Unimplemented Scope (3 เคส): ตัวคำนวณหลังบ้าน (Python Executor) ยังขาด
ฟังก์ชันสูตรคณิตศาสตร์สำหรับหมวดการหลบหนี (Flee) และคำสั่งคอมโบต่อเนื่อง
ทำให้ยังไม่รองรับแอคชั่นที่ซับซ้อนระดับนี้

Single-Prompt Baseline vs. Two-Path Architecture

เปรียบเทียบระหว่างสถาปัตยกรรมของเรา กับระบบโมเดลเดี่ยว (Single-Prompt) แบบดั้งเดิม เพื่อพิสูจน์ถึงความจำเป็นของการแยกส่วนประมวลผล (Decoupling)

The Baseline Prompt

System Prompt:

"You are a Dungeon Master running a game.

Player HP: 20, AC: 15, Zone: NEAR, Inventory: ['sword', 'shield']

Enemy HP: 10, AC: 12, Zone: NEAR

The player says: [ข้อความจำลองจาก 90 Scenarios เช่น "ฉันทิ้งกอบลิน"]

Calculate the outcome using D&D 5e logic (1d20 + stat). Determine if it hits, how much damage is dealt, and what the new HP values should be. Return ONLY a JSON block like this:

```
{
  "success": true/false,
  "narrative": "description of what happened",
  "player_new_hp": int,
  "enemy_new_hp": int,
  "player_new_inventory": []
}
```

Fairness & Constraints

- Identical Engine: ควบคุมตัวแปรให้ใช้โมเดลเดียวกันเป๊ะ (Gemini 2.5 Flash-Lite)
- Token Cost Parity (ความเท่าเทียมด้านทรัพยากร): ไม่เอาหนังสือคู่มือ D&D ความยาว 10,000 คำยัดใส่ Prompt ก็เพื่อให้ Token Cost ในการประมวลผลต่อเทิร์น ใกล้เคียงกับระบบ ของเรามากที่สุด (หากยัดกฎทั้งหมดลงไป เกมจะไม่สามารถนำไปใช้งานจริงได้เลยเพราะค่า API จะแพงมหาศาล และเกิดการ hallucination ได้ง่ายขึ้น)
- ไม่ได้มุ่งจับผิด Prompt แต่เพื่อพิสูจน์ขีดจำกัดโดยธรรมชาติของ LLM เมื่อต้องปะทะกับงานคณิตศาสตร์ที่ต้องการผลลัพธ์ตายตัว

Single-Prompt Baseline vs. Two-Path Architecture

ผลลัพธ์และการวิเคราะห์ข้อผิดพลาด

ตัวชี้วัดเชิงปริมาณ (Metrics)	Baseline (Single LLM)	DUALPATH-CORE (ของเรา)	Delta
ความแม่นยำทางคณิตศาสตร์ S_sync	0.095	0.944	+84.9 pp
ความเข้มงวดของกติกา P_ground	0	1	+100 pp

Deep Error Analysis

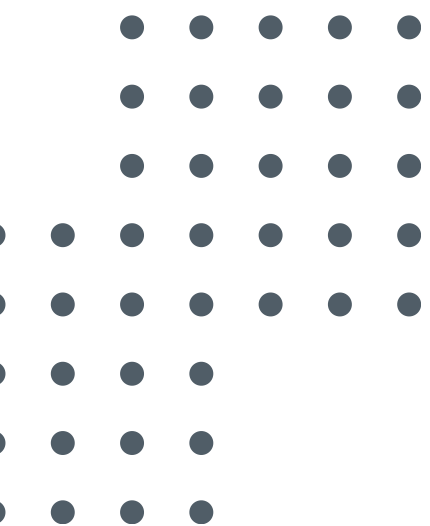
- Contextual Grounding Failure (มโนไอเทม): LLM ไม่สามารถ Map ความต้องการของผู้ใช้กับ Schema ที่มีอยู่จริงได้ เมื่อสั่ง "ดื่มยา" LLM จะ Generate ผลลัพธ์การเพิ่ม HP ขึ้นมาทันที โดยเพิกเฉยต่อความจริงที่ว่า Array กระเป๋า (['sword', 'shield']) ไม่มีไอเทมชิ้นนั้นอยู่เลย
- Action Economy Exploitation (ช่องโหว่พฤติกรรม Yes-Man): ฟังก์ชันเป้าหมาย (Objective Function) ของ LLM คือการทำตามคำสั่ง ทำให้มันขาด Guardrails เชิงลำดับเวลา เมื่อผู้เล่นสั่งจงใจโกงแบบต่อเนื่อง (เช่น "โจมตีแล้ววิ่งหนีแล้วฮีลตัวเอง") AI กลับยอมประมวลผลให้ครบทุกคำสั่งในเทิร์นเดียว
- Mathematical Hallucination (ความล้มเหลวในการผ่าเหล่า State): LLM ไม่เข้าใจตรรกะคณิตศาสตร์และการคำนวณเลือดของระบบ DnD จาก Log พบว่า AI มักคำนวณ 1d20 ผิดพลาด เพิกเฉยต่อค่าเกราะ (AC) และบางครั้งคำนวณเลือดศัตรูทะลุไปเป็นเลขติดลบ (Negative Health) ซึ่งทำให้ State Machine ของเกมล่มทันที

บทสรุปการทดสอบเชิงระบบ และความพร้อมของระบบ

100% Guardrail Enforcement: ระบบ Two-Path อดช่องโหว่การโกงเกม และสามารถปฏิเสธคำสั่งที่เป็นไปไม่ได้ (Impossible Actions) ได้ครบถ้วน

Zero Math Hallucination: การโยกย้ายสมการไปไว้ใน Python ทำให้ความผิดพลาดในการบวก/ลบ HP หายไปอย่างสิ้นเชิง

System Ready: โครงสร้างหลังบ้าน (Backend) มีความเสถียรเพียงพอที่จะนำไปให้ "มนุษย์จริง" ทดสอบแล้ว



รูปตอนให้เล่น surface

PILOT TEST

รูปตอนอัดจอ Discord

Pilot Test: Alpha Phase

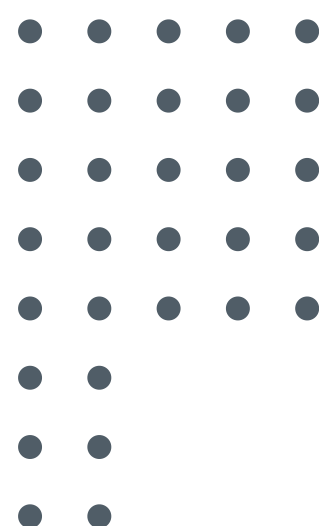
การทดสอบความถูกต้องของระบบครั้งแรกด้วยมนุษย์

Objective & Setup

รูปแบบการทดสอบ: Closed Alpha Test กับกลุ่มผู้เล่นจริง
จำนวนน้อย

เป้าหมาย: สังเกตการณ์พฤติกรรมการพิมพ์ของมนุษย์ที่คาด
เดาไม่ได้

และประเมินความสับสนไหลของระบบบรรยาย ที่ผ่านการ
ทำงานของ Two-Path Architecture



System Refinement (การเกลาความเนียนของ
ระบบ): แก้ไขปัญหาการบรรยายผิดบริบท

The Exploration Bottleneck (คอขวดระบบสำรวจ):
ผู้เล่นพบว่าการต้องพิมพ์คำสั่งที่่อยๆ ขัดกับอารมณ์การ
Roleplay นำระบบ Intent Router จากโหมดต่อสู้มา
ขยายสโคปครอบคลุมโหมดสำรวจ

การเปรียบเทียบและการควบคุมตัวแปร

การออกแบบ Baseline Prompt ที่รัดกุมที่สุด เพื่อทดสอบขีดจำกัดของ Single-Prompt LLM อย่างยุติธรรม


Prompt Pup · 4 min read

Dungeons & Dragons - ChatGPT Prompt

This prompt summons an AI-forged Dungeon Master, leading you through the fantastical realms of D&D, spinning enthralling yarns, and crafting dynamic, unbounded adventures.



Discover the power of an expert AI Dungeon Master designed to guide you through the thrilling world of **Dungeons & Dragons** with expertise in the 5th Edition ruleset.

Engaging, immersive experiences with vivid descriptions, adapting to player choices, balancing gameplay elements, and generating intriguing NPCs, all while handling dice rolls

Controlled Variables

Identical Ruleset (กฎเกณฑ์เดียวกัน): บังคับให้ Prompt ฝั่งคู่แข่งใช้กฎ Lite 5e เป๊ะๆ (เช่น Fighter 20 HP / 1 Move + 1 Action / ระบบระยะแบบ Zone-based)

Identical Initial State (จุดเริ่มต้นเดียวกัน): ล็อกสถานะผู้เล่นเริ่มต้นที่ห้องโถงกิลด์ (Guild Hall) → รับเควสต์ → ลุยดันเจี้ยน → ปะทะบอส

Mathematical Enforcement (บังคับโซ่วคณิตศาสตร์): สั่งใน Prompt ให้ AI ฝั่งคู่แข่งต้องโซ่วสมการทอยเต๋าในวงเล็บเสมอ เช่น $d20 + 2 \text{ PHYS} = 15$ vs AC 14) เพื่อให้เราตรวจสอบความถูกต้องได้โปร่งใสเหมือนระบบ Python

Pilot Test: ทดสอบผ่านผู้เล่นทั้ง 3 Personas



The Veteran

"Stress-Test กฎกติกาและ Arbiter"



The Tactician

"Stress-Test ความท้าทายและ Enemy AI"



The Casual

"Stress-Test UX และ Intent Router"

Academic Validation Methodology

5-Point Likert Scale อ้างอิงจากงานวิจัยของ Song et al. (2023), Sakellaridis, และ Jørgensen et al.

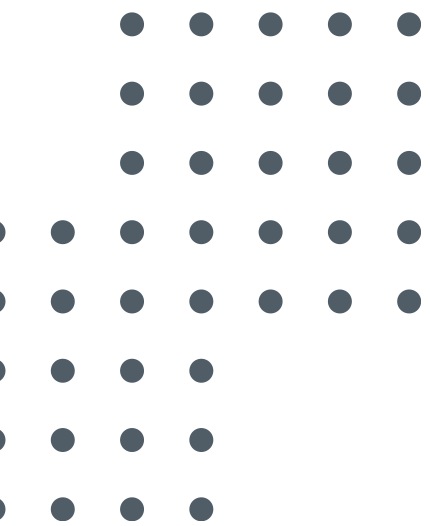
สำหรับผู้เล่นเกมทุกคน

- System Consistency (Song): AI จำของในกระเป๋าสและเลือดได้แม่นยำแค่ไหน?
- Player Autonomy (Jørgensen): รู้สึกมีอิสระในการเล่น หรือถูกบังคับเส้นทาง?
- Ease of Control (Jørgensen): พิมพ์คำสั่งเล่นเองได้ง่ายและเป็นธรรมชาติไหม?
- World Immersion (Sakellaridis): อินกับโลกและเนื้อเรื่องที่ AI สร้างแค่ไหน?
- Error Analysis: ขอจังหวะที่ประทับใจที่สุด 1 อย่าง และจังหวะที่พังที่สุด 1 อย่าง

คำถามเฉพาะทาง

- 🧙 The Veteran: Rule Reliability - AI บล็อกคุณได้ดีแค่ไหนเวลาคุณพยายามโกงกติกา?
- ♟ The Tactician: Mechanics Coherence - การต่อสู้คำนวณตัวเลขได้แฟร์และสมเหตุสมผลไหม?
- 🎮 The Casual: Narrative Adaptation & Interest - AI ปรับเนื้อเรื่องตามความกาวของคุณได้ดีไหม และเนื้อเรื่องน่าเบื่อหรือเปล่า?

User Validation



Overall Pilot Study Results

[รูปแบบในสไลด์: ใช้ Bar Chart (กราฟแท่ง) หรือ Radar Chart (กราฟใยแมงมุม)]

เอาคะแนนทั้ง 5 หัวข้อ (Consistency, Autonomy, Ease of Control, ฯลฯ) มาวางเทียบกับ 3 สี (Casual สีแดง, Veteran สีนํ้าเงิน, Tactician สีเขียว)

Average Playtime: 10-20 นาทีต่อ 1 Quest (ขึ้นอยู่กับสไตล์การอ่านและการตัดสินใจของผู้เล่น)

Pilot Results: The Casual (Non-Gamer)

score

compare

feedback

comment

analysis

key takeaway

Pilot Results: The Tactician (Boardgamer)

score

compare

feedback

comment

analysis

key takeaway

Pilot Results: The Veteran (D&D Player)

score

compare

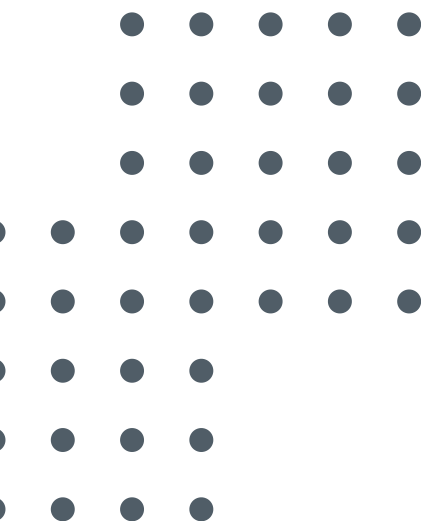
feedback

comment

analysis

key takeaway

Conclusion & Next Steps



สรุปผล

แผนงานในอนาคต

- System Optimization: แก้ไขส่วนที่เหลือนิโมดูลย่อย (เช่น การคำนวณการหนี, คอมโบ) รวมถึงทำการ Optimize ระบบให้ดีขึ้น
- Thesis Documentation (การเขียนรูปเล่ม): นำสถาปัตยกรรมที่ออกแบบไว้ ผลการทดสอบ และบทสรุปทั้งหมด มาเรียบเรียงและจัดทำเป็น รูปเล่มวิทยานิพนธ์
- Application Deployment: ตั้งใจนำสถาปัตยกรรม Game Engine หลังบ้านตัวนี้ ไปพอร์ตลงแพลตฟอร์มจริง (Web Application หรือ Mobile App)
- Public Release: เปิดให้คนทั่วไปที่ไม่มีเวลาหาแก๊งเล่นบอร์ดเกม สามารถเข้ามาสร้างตัวละคร และตะลุยดันเจี้ยนกับ AI ของเราบนเว็บไซต์ได้ตลอด 24 ชั่วโมง!



Contact Us

Institute of Field Robotics
King Mongkut's University of Technology Thonburi

A Cradle of Future Leaders in Robotics